국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

중견연구자지원사업 최종(결과)보고서

양식A101

① 부처사업명(대)	기초연구사업		보안등급(보안, 일반)	일반
② 사 업 명(중)	중견연구자지원사업		공개가능여부(공개, 비공개)	공개
③ 세부사업명(소)	중견연구(총연구비 3억 이하)			
④ 과제성격(기초, 응용, 개발)	기초	④-1 실용화 대상여부(실용화, 비실용화)		비실용화
⑤ 과 제 명	국 문	SW 품질 향상을 위한 테스팅 자동화 및 디버깅 자동화 기술		
	영 문	Automated SW testing and debugging techniques for improving SW quality		
⑥ 주관연구기관	한국과학기술원			
⑦ 협동연구기관				
⑧ 주관연구책임자	성 명	김문주	직급(직위)	부교수
	소속부서	전산학부	전 공	소프트웨어공학

⑨ 연구개발비 및 참여연구원수 (단위: 천원, M·Y)

년 도	정부출연금 (A)	기업체부담금			정부외 출연금 (B)	상대국 부담금 (F)	합계 G=(A+B+E)	참여 연구원수
		현금 (C)	현물 (D)	소계 E=(C+D)				
1차년도	99000			0			99,000	6
2차년도	82500			0			82,500	10
3차년도	90750			0			90,750	10
4차년도	24750			0			24,750	8
5차년도				0			0	
합계	297,000	0	0	0	0	0	297,000	34

⑩ 총연구기간	2016. 06. 01 ~ 2019. 05. 31 (36개월)
⑪ 다년도협약연구기간	기재하지 않음
⑫ 당해연도연구기간	2019. 03. 01 ~ 2019. 05. 31 (3개월)

⑬ 참여기업	중소기업수	대기업수	기타	계
				0

⑭ 국제공동연구	상대국연구기관수	상대국연구개발비	상대국연구책임자수

관계 규정과 모든 지시사항을 준수하면서 국가연구개발사업에 따라 수행 중인 연구개발과제의 최종보고서를 붙임과 같이 제출 합니다.

2019년 6 월 26 일

주관연구책임자 : 김문주

주관연구기관장 : 신성철

※ 전자접수이므로 주관연구책임자 및 주관연구기관장 서명(인, 직인)은 생략

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 연구결과 요약문 〉

양식A202

연구개요	본 연구는 SW 오류를 자동으로 발견하는 체계적이고 과학적인 기술 개발을 통한 SW품질향상을 목표로 한다. 효과적인 자동 테스팅 및 자동 디버깅 기술을 개발하기 위해, 본 연구는 동적 심볼릭 실행 기술 (dynamic symbolic execution)과 SW변형기술 (software mutation analysis)을 활용하여 다음의 3가지 세부연구 주제를 다룬다. ● SW유닛 테스트 환경 자동 생성 기술: 다양하고 실제적이며 간결한 SW유닛 테스팅 환경을 자동으로 생성한 후, 동적 심볼릭 실행을 적용하여 SW유닛 테스팅을 100% 자동화하는 기술 ● SW변형 기반 자동 테스트 기술: SW유닛 테스팅 환경의 제약점 극복을 위해, 대상 SW유닛 자체를 변형하여 효과적인 테스트를 자동으로 생성하는 기술 ● SW 디버깅 정보 자동 생성 기술: 수많은 테스트 케이스를 분석하여, 버그 위치를 자동으로 탐색하고 버그 수정에 필요한 정보를 자동으로 생성하는 기술
연구 목표대비 연구결과	본 연구에서는 대상SW의 다양한 동작을 관찰하기 위해 동적 심볼릭 실행 기법(dynamic symbolic execution)과 SW변형기법 (mutation analysis)을 기반으로 SW유닛 테스팅 환경을 자동으로 생성하고, SW오류 검출 및 디버깅을 자동으로 수행하는 연구를 진행하여 다음 연구결과를 얻을 수 있었다. ● SW유닛 테스트 환경 자동 생성 기술: - 동적 분석을 활용한 조건부 확률 기반 함수간 연관도 측정 메트릭 기술 개발 - 유닛 테스트 정확도 향상을 위한 함수 연관도에 기반한 심볼릭 함수 호출 문맥 기술 개발 - 세계 최고 수준의 오류 검출 능력 (91% 오류 검출) 및 정확도 (거짓:참 경보 비율 4.5:1) 달성 ● SW변형 기반 자동 테스트 기술: - 오픈소스 C 프로그램 변형 분석 도구 MUSIC 개발 (https://github.com/swtv-kaist/MUSIC) - 변형 프로그램으로부터 커버리지 향상을 위한 가이드포스트 생성 및 SW 테스트 자동 생성 기술 개발 - 오픈소스 C 프로그램 대상 커버리지 평균 14.1%p 향상 ● SW 디버깅 정보 자동 생성 기술: - SW 변형 분석과 관찰 기반 분석을 결합한 정확도 높은 버그 위치 탐색 기술 개발 - 기계학습 기반 SW 디버깅 정보 생성 기술 개발 - 세계 최고 수준의 버그 위치 탐색 정확도 (평균 5.9% 구문 분석) 달성
연구개발결과의 중요성	본 연구는 SW 테스팅 및 디버깅 전반의 자동화와 고도화를 통해 SW품질 향상, SW테스팅 소요인력 절감, SW테스팅 기간단축으로 인한 제품개발 기간단축 등을 통하여 제품 품질 및 개발 효율을 향상시킬 수 있다. 본 연구 결과는 기존에 달성하지 못한 세계 최고 수준의 오류 탐지 능력과 정확도를 달성하였을 뿐 아니라, 실제 개발 환경에 적용하여 SW 테스팅 비용을 줄이고 품질 향상에 기여하였음을 보였다. 특히, 현대모비스와의 산학협력을 통해 본 연구 결과가 차량 SW 테스팅 시간을 50~70% 단축시킬 수 있음을 실증적으로 입증하여 현대모비스의 차량 SW 테스트 프로세스에 본 연구 결과가 전사적으로 도입되는 성과를 달성하였다. 학문적/연구적 측면에서는, 기존 SW테스팅 자동화 연구 범위를 테스트케이스 생성 수준에서 테스팅 환경 분석 및 자동 생성, 디버깅 자동화 분야까지 확장한 세계적으로 선도적인 연구이므로, 후속 관련 연구들이 활성화될 것을 기대한다. 특히, SW변형 기반 테스트 생성 기법의 경우, 기존 테스팅의 한계를 극복하는 세계 첫 번째의 독창적인 시도인 만큼, SW 변형 기법을 새로운 관점/용도로 활용하는 다양한 후속 연구들이 파생되어 새로운 연구주제들이 도출되리라 기대한다.
중심어	소프트웨어 자동화 테스팅 / 프로그램 변형기법 / 동적심볼릭기법 오류위치 추적기법 / 유닛 테스팅 /

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 목 차 〉

1. 연구개발과제의 개요 ····· 04
2. 연구수행내용 및 연구결과 ····· 05
3. 연구개발결과의 중요성 ····· 14
4. 참고문헌 ····· 15
5. 연구성과 ····· 17
<연구책임자 대표적 연구실적> ····· 26

<별첨> 주관연구기관의 자체평가 의견서

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

1. 연구개발과제의 개요

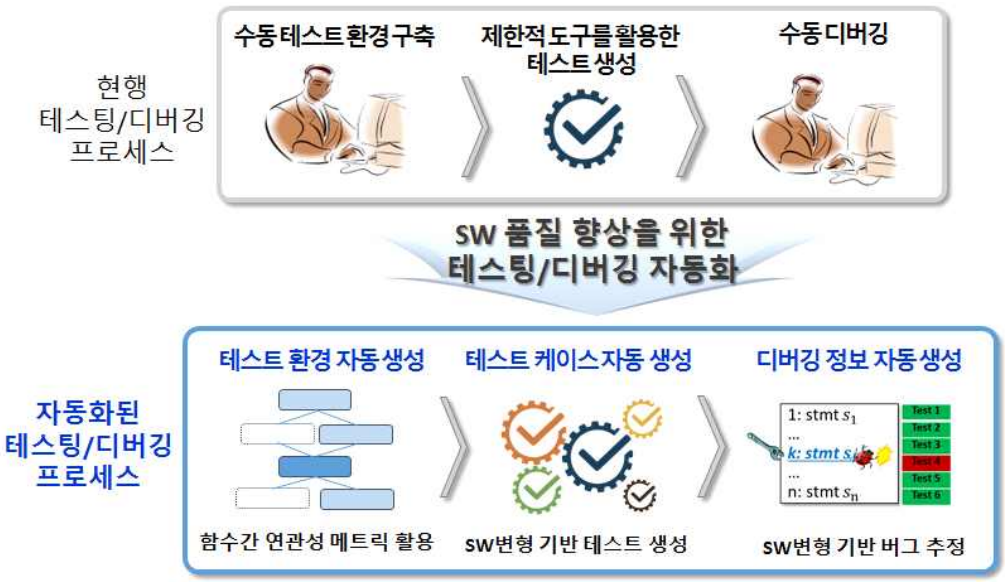


그림 1. 제안한 연구를 통한 SW 테스팅/디버깅 프로세스 개선

- **SW유닛 테스트 환경 자동 생성 기술**: 다양하고 실제적이며 간결한 SW유닛 테스팅 환경을 자동으로 생성한 후, 동적 심볼릭 실행을 적용하여 SW유닛 테스팅을 100% 자동화하는 기술

  테스트 드라이버, 스텁 (stub), 테스트 케이스 등 SW유닛 환경은 SW 유닛 테스트에 큰 영향을 끼치는 만큼, 다양하고 실제적인 SW 유닛 환경을 생성하는 기술을 연구한다. 이를 위해 대상 SW유닛과 연관성이 높은 다른 SW유닛의 코드를 포함하여 동적 심볼릭 실행을 수행하여, 대상 SW유닛에 다양하고 실제적인 환경(context)를 제공하여 대상 SW유닛의 다양하고 실제적인 동작을 테스트한다.

- **SW변형 기반 자동 테스트 기술**: SW유닛 테스팅 환경의 제약점 극복을 위해, 대상 SW유닛 자체를 변형하여 효과적인 테스트를 자동으로 생성하는 기술

  SW유닛 테스팅 환경의 제약점을 극복하는 또 다른 방법으로, 대상 SW유닛 자체를 변형한 후 테스트한 동작들 중 원본 SW유닛에서 오류를 일으킬 수 있는 동작들을 선별한다. 그리고 그러한 동작들로부터 얻은 정보를 바탕으로 원본 SW에서 발생 가능한 오류 시나리오를 생성한 후, 오류 시나리오를 재현 할 수 있는 테스팅 환경을 자동으로 생성한다.

- **SW 디버깅 정보 자동 생성 기술**: 수많은 테스트 케이스를 분석하여, 버그 위치를 자동으로 탐색하고 버그 수정에 필요한 정보를 자동으로 생성하는 기술

  대상SW의 구문을 하나씩 변형시킨 변형SW들의 테스트케이스의 결과와 원본SW의 테스트케이스 결과의 차이를 바탕으로 버그 위치를 정확하게 찾아내고, 버그 위치에 변형이 생성된 변형SW들의 동작을 관찰하여 버그 수정에 필요한 정보를 추출하여 디버깅에 사용한다. 또한 수많은 변형 SW들이 생성되어 이들을 테스트하는 시간이 크므로, 디버깅 자동화에 대한 효과 및 효율이 높은 변형연산자 및 테스트케이스를 선별하여 적용한다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 연구수행내용 및 연구결과

(1) SW유닛 테스트 환경 자동 생성 기술

다양하고 실제적인 유닛 테스팅 환경 자동 생성

- 동적 분석을 활용한 조건부 확률 기반 **함수간 연관도 (relevance) 측정 메트릭** 개발
  - 알고리즘 1의 Step 1과 Step 2 참고
- 연관도 높은 함수들을 포함하는 확장 유닛 테스트 환경 자동 생성 기술 개발
  - 알고리즘 1의 Step 3 참고
- 오픈소스 C 프로그램 대상으로, 연관도 측정 메트릭을 활용한 확장 유닛 테스트가, 오류를 많이 검출하면서 거짓 경보를 줄임을 실험적으로 보임
  - "나) 연구 기술 적용 실험 결과" 참고
- 연구결과를 응용하여 현대모비스와 차량용SW 테스팅 도구 MAIST를 공동 개발하여 테스팅 비용 50% 이상 감소
  - 대표적 연구실적 4번, 5번 및 "나) 연구 기술 적용 실험 결과" 마지막 문단 참고

효과적인 오류 검출을 위한 새로운 동적 심볼릭 실행 탐색 기법

- **테스트 대상 유닛 함수의 분기문에 높은 우선순위를 부여**하는 효과적인 동적 심볼릭 실행 탐색 기법 개발
  - 알고리즘 1의 Step 4 참고
- 오픈 소스 C 프로그램을 대상으로, 높은 오류 검출 능력 및 낮은 거짓 경보 비율 입증
  - "나) 연구 기술 적용 실험 결과" 참고

유닛 테스팅 정확도 향상을 위한 거짓 경보 감소 기술

- **함수 연관도에 기반한 심볼릭 함수 호출 문맥** (symbolic calling context) 생성 기술 개발
  - 알고리즘 1의 Step 3 참고
- 심볼릭 함수 호출 문맥을 활용한 거짓경보 제거기술 개발
  - 알고리즘 1의 Step 5 참고
- 오픈 소스 C 프로그램을 대상으로, 심볼릭 함수 호출 문맥을 활용한 거짓 경보 제거 기술이, 오류 탐지 능력은 거의 그대로이면서, 거짓 경보를 효과적으로 제거할 수 있음을 보임
  - "나) 연구 기술 적용 실험 결과" 참고
- 현존 세계최고 기술보다 우수한 거짓경보 제거 성능 달성 (실제 탐지 오류 당 거짓 경보 개수 4.5 vs 5.7/5.9/6.3)
  - "나) 연구 기술 적용 실험 결과" 마지막에서 두번째 문단 참고

가) 연구 기술 요약 설명 (대표적 연구실적 2번인 ICSE 2018 발표 논문 참고)

SW 유닛 테스팅을 사용하여 효과적으로 오류를 검출하고 거짓 경보를 줄이기 위한 SW 유닛 테스트 환경 자동 생성 기술을 개발하였다. 기존 자동화된 유닛 테스트 환경 자동 생성 기술은 테스트 대상 함수를 호출하는 caller 함수와 테스트 대상 함수가 실행 중에 호출하는 callee 함수의 정보를 활용하지 못해 거짓 경보, 즉 유닛 테스트 환경에서는 실행 가능하지만 실제 프로그램 수행 도중에는 실행 불가능한 경보가 다수 발생한다. 본 연구에서 개발한 기법은 테스트 대상 함수와 높은 의존도를 갖는 caller 함수와 callee 함수의 정보를 활용하여 거짓 경보를 최소화하는 기술을 개발하였다. 기존에 존재하는 시스템 테스트를 실행하여 함수와 함수 사이의 의존도를 동적으로 분석하고 의존도 정보를 활용하여 거짓 경보 제거를 효과적으로 수행한다. 그림 2는 효과적인 오류 검출을 위한 자동 유닛 테스트 생성 기법의 개요를 나타낸다. SW 유닛 테스트 환경 자동 생성 기술은 알고리즘 1에 보이듯이, 5 단계로 구성되어 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

○ **Step 1: 시스템 테스트 실행 결과로부터 함수 커버리지 프로파일 생성**
주어진 시스템 테스트 케이스를 사용해서 타겟 프로그램을 실행하고 각각의 시스템 테스트에서 함수 호출 관계를 caller-callee 함수 쌍으로 추출한다.

○ **Step 2: 함수 사이의 의존도 계수 계산**
Step 1에서 추출한 함수 호출 관계를 활용하여 조건부 확률로 함수 의존도를 계산한다.

○ **Step 3: 확장 유닛 및 symbolic calling context 생성**
타겟 함수 f와 다른 함수의 조건부 확률 기반 의존도를 계산하여 확장 유닛과 symbolic calling context를 계산한다. 확장 유닛은 유닛 테스트시 실제 코드를 실행할 callee 함수들의 집합으로 callee 함수의 context를 제공하고 symbolic calling context 는 거짓 경보 제거 휴리스틱에서 활용할 caller 함수의 call path 정보를 나타낸다.

○ **Step 4: Concolic 테스팅 실행을 통한 유닛 테스트 입력 생성**
Step 3에서 생성한 확장 유닛을 사용하여 Concolic 테스트를, 테스트 대상 유닛 함수의 분기문에 높은 우선순위를 부여하여 수행하고 단언문 위반 경보(assert violation)가 발생한 심볼릭 실행 경로(symbolic execution path)를 기록한다.

○ **Step 5: 거짓 경보 필터링**
Step 4에서 발생한 단언문 위반 경보 중에서 symbolic calling context의 조건을 만족하지 않는 경보를 제거하고 제거되지 않은 경보를 사용자에게 보고한다.

알고리즘 1. SW 유닛 테스트 환경 자동 생성 기술

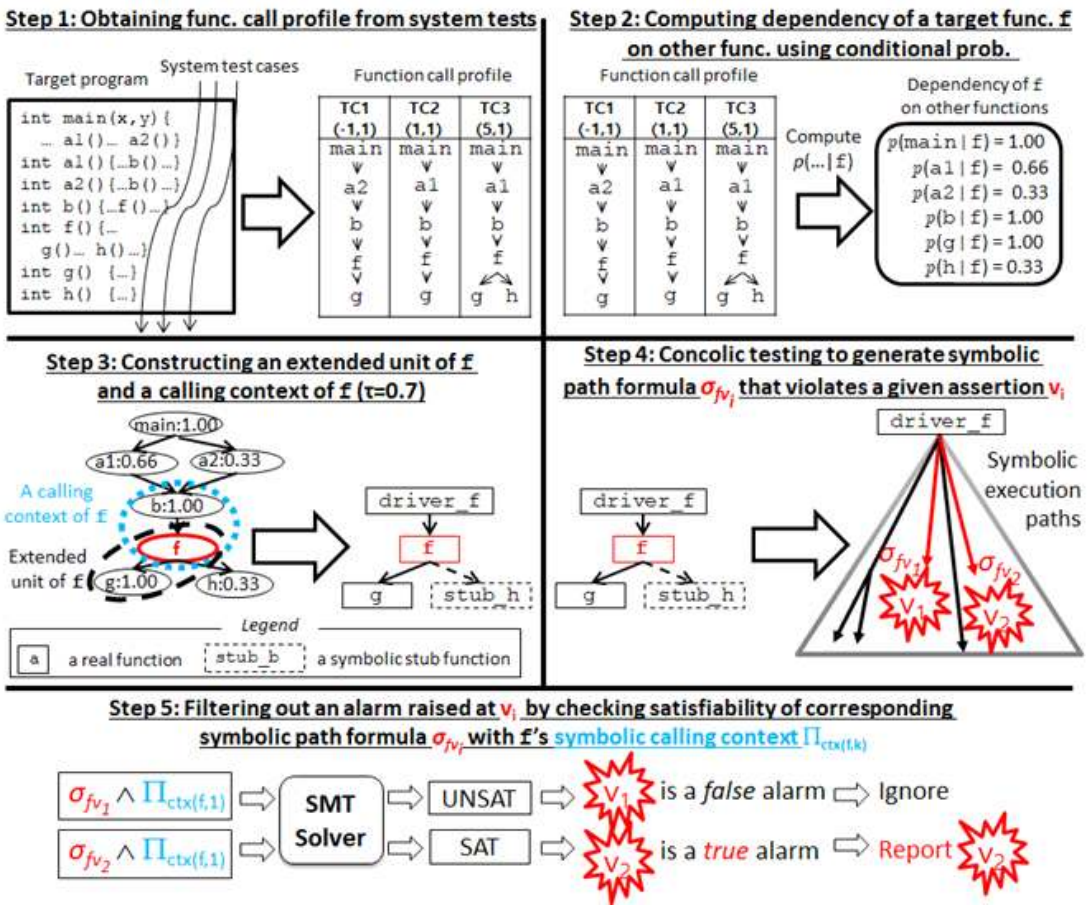


그림 2. 효과적인 오류 검출을 위한 자동 유닛 테스트 생성 기법 개요

나) 연구 기술 적용 실험 결과 (대표적 연구실적 2번인 ICSE 2018 발표 논문 참고)
제안하는 자동 유닛 테스트 생성 기법이 실제로 오류를 효과적으로 탐지하고 거짓 경보를 제거하는지

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

확인하기 위해 SIR/SPEC2006 벤치마크에 포함된 15개 C 프로그램을 대상으로 실험을 수행하였다. 15개 프로그램의 평균 크기는 55328.6 줄이며 15개 프로그램에 포함된 타겟 크래시 오류 수는 총 67개였다.

효과적인 오류 검출을 위한 자동 유닛 테스트 생성 기법을 평가하기 위해 다음 두 연구 질문을 만들고 실험을 통해 연구 질문에 답하였다.

- 연구 질문 1(오류 탐지 능력): 제안한 효과적인 오류 검출을 위한 자동 유닛 테스트 생성 기법이 다른 Concolic 유닛 테스팅 기법에 비해 얼마나 많은 오류를 탐지할 수 있는가?

- 연구 질문 2(오류 탐지 정확도): 제안한 효과적인 오류 검출을 위한 자동 유닛 테스트 생성 기법이 다른 Concolic 유닛 테스팅 기법에 비해 얼마나 거짓/참 경보 수 비율을 감소시킬 수 있는가?

비교 대상 테크닉은 다음과 같다.

- 심볼릭 유닛 테스팅(SUT): 타겟 함수 f의 모든 파라메터와 f가 사용하는 전역변수를 심볼릭 입력으로 생성하고 f가 호출하는 모든 함수를 심볼릭 입력 값을 리턴하는 스텁 함수로 대체하는 가장 기본적인 Concolic 유닛 테스팅 테크닉
- CONBOL(CONcrete+symBOLic): 심볼릭 유닛 테스팅 테크닉에 거짓 경보를 줄이기 위한 휴리스틱과 경보 우선순위 규칙을 사용한 Concolic 유닛 테스팅 테크닉
- 동적 함수 상관관계를 활용한 자동 유닛 테스트 생성: 본 연구에서 제안한 기법

표 1. 각 유닛 테스팅 테크닉이 탐지한 크래시 오류 수 및 참/거짓 경보 비율

타겟 프로그램	타겟 크래시 오류 수	심볼릭 유닛 테스팅		CONBOL		제안하는 기법	
		탐지 오류 수	거짓/참 경보 비율	탐지 오류 수	거짓/참 경보 비율	탐지 오류 수	거짓/참 경보 비율
bash	6	5	96.8	4	33.3	5	3.6
flex	2	2	71.0	1	23.0	1	6.0
grep	5	3	40.0	4	8.0	4	3.3
gzip	2	2	16.5	2	3.5	2	2.5
make	3	3	221.3	2	49.5	3	3.0
sed	2	2	15.5	2	5.0	2	2.5
vim	6	5	181.2	3	67.7	5	5.0
perl	6	6	65.3	4	46.8	6	9.5
bzip2	2	2	17.0	2	5.5	2	5.0
gcc	15	14	144.7	11	50.3	14	5.6
gobmk	5	4	197.8	3	45.3	5	7.8
hmmer	3	3	54.0	2	25.5	3	4.0
sjeng	2	2	54.0	2	7.0	2	4.0
libquantum	3	3	18.3	2	4.5	3	1.7
h264ref	5	5	46.4	4	8.5	4	4.3
총합	67	61	82.7	48	25.6	61	4.5

표 1은 비교 대상 유닛 테스팅 테크닉이 탐지한 크래시 오류 수 및 참/거짓 경보 비율을 나타낸다. 제안하는 기법은 타겟 크래시 오류 67개 중에 61개를 탐지하여 심볼릭 유닛 테스팅 기법과 더불어 가장 높은 오류 탐지 능력을 보였으며 CONBOL은 67개중 48개의 오류만 탐지하여 가장 낮은 오류 탐지 능력을 보였다. 따라서 제안하는 동적 상관관계를 활용한 자동 유닛 테스트 생성 기법은 실제 오류를 효과적으로 탐지할 수 있음을 보였다.

동적 상관관계를 활용한 자동 유닛 테스트 생성 기법은 1개의 실제 오류에 4.5개의 거짓 경보가 발생하

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

여 가장 낮은 거짓/참 경보 비율(1개의 실제 오류당 발생하는 거짓 경보 수 비율)을 보였다. 심볼릭 유닛 테스팅의 경우 거짓 경보를 제거하기 위한 기법을 전혀 적용하지 않았기 때문에 가장 높은 참/거짓 경보 비율(1개의 오류당 평균 82.7개)을 보였다. CONBOL의 경우 거짓 경보를 제거하기 위한 휴리스틱이 적용되면서 25.6의 거짓/참 경보 비율을 보여 심볼릭 유닛 테스팅보다 거짓/참 경보 비율이 크게 감소했지만 여전히 높은 수준의 참/거짓 경보 비율을 보였다.

본 연구 성과인, **동적 상관 관계를 활용한 자동 유닛 테스트 생성 기법은 현존하는 세계 최고의 유닛 테스트 자동 생성 기법들보다 낮은/우수한 거짓/참 경보 비율을 보였다.** Java 프로그램 언어를 대상으로 하는 Randoop 과 Evosuite 유닛 테스트 생성 기법은 객체지향 언어의 캡슐화 언어 특성을 컨텍스트 제공 힌트로 사용할 수 있음에도 불구하고 5.9, 6.3의 거짓/참 경보 비율을 보여 제안하는 기법보다 더 많은 거짓 경보를 발생시켰다. C 프로그램 언어를 대상으로 하는 UC-KLEE의 경우 사용자가 거짓 경보를 제거하기 위한 힌트를 수작업으로 제공해줬음에도 불구하고 사용자 힌트를 전혀 활용하지 않는 제안 기법보다 더 높은 5.7의 거짓/참 경보 비율을 보였다.

또한, 현대모비스와 공동으로 함수간 연관도를 응용한 차량용 SW 자동 테스팅 도구 MAIST를 개발하여, 차량용 SW IBU (integrated body unit) 수작업 테스팅에 소요되는 시간을 절반이상 단축했다 (대표적 연구실적 4번인 ICSE 2019 발표 논문 및 대표적 연구실적 5번 보도 자료 참고).

(2) SW변형 기반 자동 테스트 기술

SW변형 기반 테스팅 기술 정립
- 변형 프로그램의 (mutated program) 테스트 커버리지 및 오류 검출 능력 증가 가능성 조사
  - “가) 연구 기술 요약 설명” 참고
- 변형 프로그램으로부터 테스트 커버리지를 증가시킬 수 있는 실행 조건인 **가이드포스트 생성 기술** 개발
  - 알고리즘 2의 Step 2 참고
- 오픈소스 C 프로그램을 대상으로, 변형 프로그램을 활용한 원본 프로그램의 테스트 커버리지 증가 입증
  - “나) 연구 기술 적용 실험 결과” 참고

SW 변형 기반 테스팅 프레임워크에 필요한 SW 변형 분석 도구 및 인프라 개발
- **C 프로그램 변형 분석 도구 MUSIC** (MUtation analySIs tool with high Configurability and Extensibility) 오픈소스 개발
  - 최신 C11 표준을 따르는 C 프로그램 적용 가능
  - C 프로그램을 위해 설계된 73개 변형 연산자 구현
  - 효율적인 변형 분석을 위한 상세한 환경 설정 기능 제공
  - 세부목표 증빙 2번, 알고리즘 2의 Step 1, 및 “나) 연구 기술 적용 실험 결과” 첫문단 참고

SW변형 기반 테스팅 프레임워크 고도화 및 사례연구
- 변형 프로그램에서 생성한 **가이드포스트를 활용한 SW 테스트 자동 생성 기술** 개발
  - DeMiner 기술 개발 (“가) 연구 기술 요약 설명” 참고)
- 오픈소스 C 프로그램을 대상으로 적용하여 SW 변형 기반 테스팅 프레임워크를 통한 테스트 커버리지 향상 입증
  - “나) 연구 기술 적용 실험 결과” 참고

가) 연구 기술 요약 설명 (대표적 연구실적 3번인 ICST 2018 발표 논문 참고)

SW 변형 기반 자동 테스트 기술은 다른 공학분야에서 제품 및 구조물의 내구도를 테스트 하기 위한 파괴 테스트 기법에서 착안하여 SW를 변형/파괴하고 변형/파괴된 SW의 실행 결과를 분석하여 SW 오류를 탐지하는 기법이다. 그림 3의 좌측 사진은 자동차의 내구도를 테스트 하기 위해 차량을 파괴하면서 파괴될 때의 충격량과 내구 손상을 측정하는 파괴 테스트의 사례이다. 이에 착안하여 SW 변형 기반 자동 테스트 기술은 주어진 테스트 대상 SW v0를 변형하여 v1, v2, ..., vn의 변형된 SW를 생성하고 각 변형된 SW를 테스트한 테스트 결과 R1, ..., Rn을 생성한다. 그 후 변형 SW의 테스트 결과 R1, ..., Rn을 분석하여 테스트 대상 v0를 테스트했을 때 발견하지 못한 오류를 변형 SW 결과를 분석해서 발견할 수 있다.

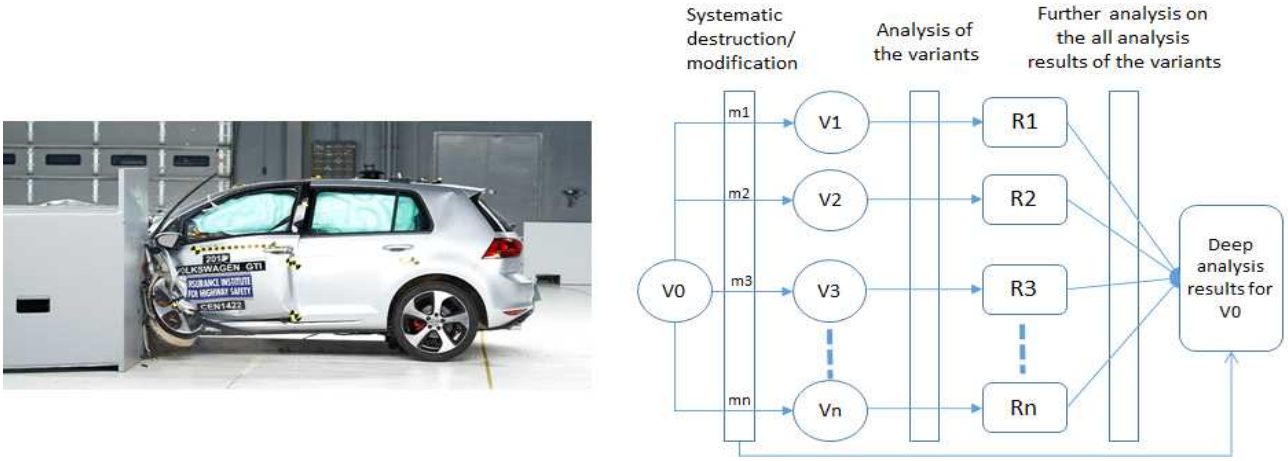


그림 3. 자동차 내구 테스트를 위한 파괴 테스트 사례(좌)와 SW 오류 탐지를 위한 파괴 테스트 개요(우)

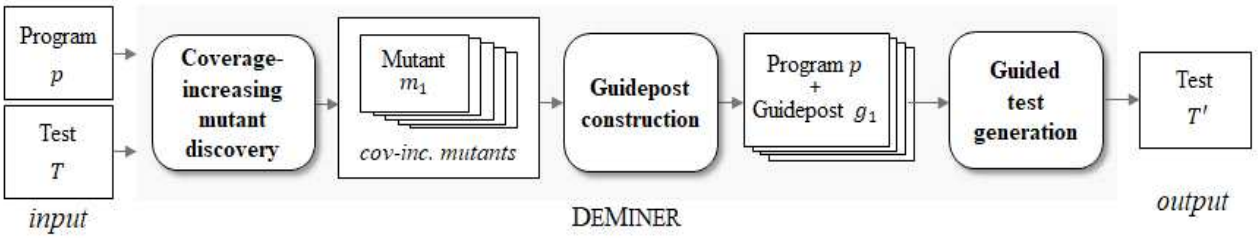


그림 4. SW변형 기반 자동 테스트 생성 기술 개요

그림 4는 SW 변형 기반 자동 테스트 기술을 구체적으로 구현한 DeMiner (guiDEed test generation using MutatIoN ExploRation)기술의 개요를 나타낸다. 먼저 테스트 대상 프로그램 P와 테스트 집합 T를 입력으로 받아서 다음 3단계를 수행하여 SW 변형 기반 자동 테스트 기술을 아래의 알고리즘 2와 같이 적용한다.

○ **Step 1: 커버리지가 증가한 변형 프로그램 탐색**
DeMiner 기술은 먼저 테스트 대상 프로그램 P를 변형하여 다양한 변형 프로그램을 생성하고 (기존 변형 분석 도구들의 약점을 극복한 MUSIC을 사용하여 변형 프로그램 생성) 각각의 변형 프로그램 m1, ..., mn 에 테스트 집합 T를 실행하여 테스트 커버리지(테스트가 실행한 SW 구문)를 측정한다. 변형 기존 프로그램 P에 테스트 집합 T를 실행했을 때의 테스트 커버리지와 변형 프로그램 mi에 테스트 집합 T를 실행했을 때의 테스트 커버리지를 비교하여 변형 프로그램에서 기존에 실행하지 못한 구문을 실행했으면, 커버리지가 증가한 변형 프로그램 mi를 커버리지가 증가한 변형 프로그램으로 간주한다.

○ **Step 2: 가이드포스트 (guidepost) 생성**
Step 1에서 찾은 커버리지가 증가한 변형 프로그램의 테스트 실행 정보를 분석하여 테스트 대상 프로그램 P의 기존에 실행하지 못한 구문을 실행하기 위한 가이드포스트 구문을 생성한다. 만약 가이드포스트를 만족할 수 있는 테스트 입력 값을 생성하면 해당 테스트를 실행하여 기존에 실행하지 못한 구문을 실행하여 테스트 커버리지를 높일 수 있다.

○ **Step 3: 가이드포스트를 만족하는 테스트 자동 생성**
Step 2에서 생성한 가이드포스트를 만족하는 테스트 입력 값을 Concolic 테스트 기법을 사용하여 자동으로 생성한다. Concolic 테스트 기법을 사용하여 가이드포스트를 만족하는 테스트 입력을 높은 우선순위로 생성하기 위해 Concolic 테스트 탐색 기법을 최적화하였다.

알고리즘 2. SW 변형 기반 자동 테스트 기술

나) 연구 기술 적용 실험 결과 (대표적 연구실적 3번인 ICST 2018 발표 논문 참고)
개발한 SW 변형 기반 자동 테스트 기술의 효과적 오류 탐지를 위한 커버리지 증가를 확인하기 위해 3

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

개 C 오픈 소스 프로그램(Busybox-ls, Busybox-printf, GNU find)을 대상으로 실험을 수행하였다. 테스트 대상 프로그램 크기는 평균 1396.3 줄이며 평균적으로 44.3개의 개발자가 작성한 시스템 테스트가 있고 이를 실행하여 69.0%의 구문을 커버하였다. 기존 C 프로그램 변형 분석 도구인 Milu와 Proteum은 테스트 대상 오픈 소스 프로그램의 변형 프로그램을 생성하지 못했으나 MUSIC은 변형 프로그램을 성공적으로 생성하였다.

연구질문 1. 변형 프로그램을 실행하여 얼마나 많은 구문을 새로 실행할 수 있는가?
연구질문 2. 변형 프로그램 분석으로 생성한 가이드포스트를 활용하여 테스트 대상 프로그램에서 얼마나 많은 구문을 새로 실행할 수 있는가?

표 2. 변형 프로그램 생성 및 실행 결과

타겟 프로그램	총 변형 프로그램 생성 수	커버리지가 증가한 변형 프로그램 수	변형 프로그램 실행으로 커버한 총 구문 수 및 비율 (%)	변형 프로그램에서 새로 실행한 구문 수 및 비율 (%)
Busybox-ls	17313	1280	363 (89.9%)	106 (26.2%)
Busybox-printf	5728	118	163 (96.4%)	23 (13.6%)
GNU find	5501	172	2505 (69.4%)	313 (8.7%)
평균	9514.0	523.3	1010.3 (85.2%)	147.3 (16.2%)

표 2는 연구질문 1에 답하기 위한 변형 프로그램 생성 및 실행 결과로, **변형 프로그램은 기존 프로그램이 실행 못한 구문 실행에 효과적이기에, 가이드포스트 생성에 필요한 정보를 제공할 수 있음을 확인했다.** 타겟 프로그램당 평균 9514.0개의 변형 프로그램을 생성하였고, 그 중 523.3개의 변형 프로그램에서 커버리지가 증가했다. 변형 프로그램을 실행한 결과, 타겟 프로그램 당 평균 1010.3개의 구문을 실행하였다 (전체 구문 수의 85.2%). 즉, 원본 프로그램에서 실행한 구문 수와 변형 프로그램 구문에서 실행한 구문 수를 비교하면 평균적으로 **전체 프로그램 구문의 16.2%에 해당하는 147.3개의 구문이 변형 프로그램에서 추가 실행**되었다.

표 3. SW 변형 기반 자동 테스트 생성 기술 적용 결과

타겟 프로그램	가이드포스트 생성 수	변형 프로그램 실행으로 커버한 총 구문 수 및 비율 (%)	변형 프로그램에서 새로 실행한 구문 수 및 비율 (%)
Busybox-ls	404	348 (86.1%)	91 (22.5%)
Busybox-printf	169	161 (95.3%)	21 (12.4%)
GNU find	3616	2458 (68.0%)	266 (7.4%)
평균	1396.3	989.0 (83.1%)	126.0 (14.1%)

표 3은 연구질문 2에 답하기 위한 실험 결과로, **SW 변형 기반 자동 테스트 생성 기술이 원본 프로그램의 테스트 커버리지를 증가시키는데 효과적이며, 따라서 오류 탐지 능력을 향상할 수 있음을 확인했다.** 테스트 커버리지가 증가할수록 기존 테스트가 발견하지 못한 오류를 발견할 확률이 높아지기 때문에 테스트 커버리지를 높이는 것이 오류 탐지 능력을 높이는데 도움이 된다. 변형 프로그램 실행 정보를 분석하여 프로그램 당 평균 1396.3개의 가이드포스트를 생성하였다. 가이드포스트를 만족하는 테스트를 생성하도록 Concolic 테스트를 수행한 결과 평균적으로 전체 구문의 83.1% 에 해당하는 989.0개의 구문을 최종적으로 실행할 수 있었다. **SW 변형 기반 자동 테스트 생성 기술이 추가로 커버한 원본 프로그램의 구문 수는 전체 구문 수의 14.1%에 해당하는 126.0개의 구문**이다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

(3) SW 디버깅 정보 자동 생성 기술

SW변형 분석 기반 버그 위치 탐색 기술

- SW 변형 분석 (mutation analysis) 기반 오류 위치 탐색 기법 (mutation-based fault localization (MBFL)) 공식과 SW 관찰 기반 오류 위치 탐색기법 (spectrum-based fault localization (SBFL)) 공식을 결합하여 각 기법의 단점을 보완하는 기술 개발
  - "가) 연구 기술 요약 설명" 참고
- 오픈소스 C 프로그램을 대상으로, MBFL 과 SBFL 공식을 결합해서 오류 위치 탐색 정확도를 높일 수 있음을 보임
  - "나) 연구 기술 적용 실험 결과" 참고

효율적인 SW 변형 기반 버그 위치 탐색을 위한 변형 연산자/테스트 케이스 선별

- SW 변형 기반 버그 위치 탐색 (MBFL)의 정확도는 높이고 실행 시간은 줄이는 변형 연산자/변형 프로그램 선택 기법 개발
  - 알고리즘 3의 Step 2 참고
- SW 변형 기반 버그 위치 탐색 (MBFL)의 정확도는 높이고 실행 시간은 줄이는 테스트 케이스 선택 기법 개발
  - 알고리즘 3의 Step 2 참고
- 오픈소스 C/Java 프로그램의 8개 버그를 대상으로, **오류 위치 탐색 정확도는 크게 떨어지지 않고 (최대 2배 저하), 실행 속도는 10배 빨라짐을 입증**
  - "나) 연구 기술 적용 실험 결과" 참고

변형된 SW를 활용하여 SW버그 패치에 필요한 정보 생성

- 기계학습을 활용하여, SW 변형 분석 기반 버그 위치 탐색 기법 (MBFL)의 휘처 (feature) 들과 SW 관찰 기반 버그 위치 탐색 기법 (SBFL)의 휘처를 결합된 SW 디버깅 정보 생성 기술 개발
  - 알고리즘 3의 Step 3 참고
- 오픈소스 C 프로그램을 대상으로, 제안한 기계학습 기반 SW 디버깅 정보 생성 기술이, 디버깅에 필요한 오류 위치를 정확하게 추정할 수 있음을 보임
  - "나) 연구 기술 적용 실험 결과" 참고

가) 연구 기술 요약 설명 (세부 목표 증빙 1번 참고)

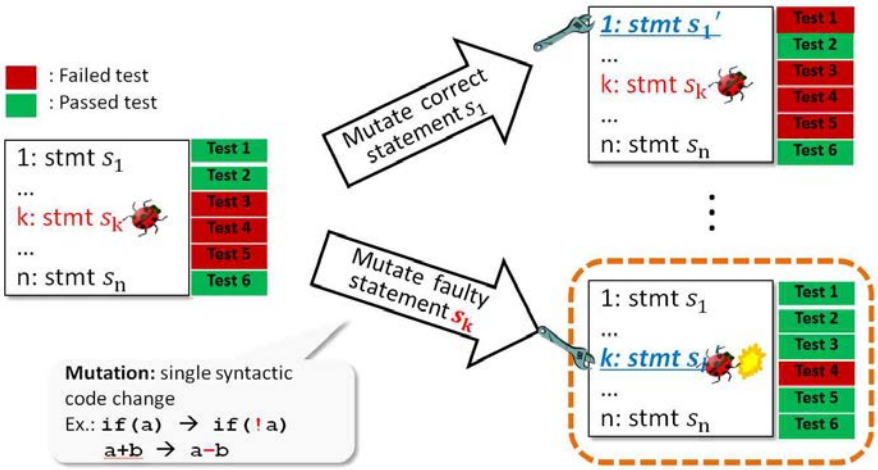


그림 5. SW 변형 분석 기반 SW 디버깅 정보 자동 생성 기술 개요

SW 오류가 발생해서 디버깅을 수행할 때 대부분의 개발자가 수작업으로 오류 발생 구문부터 실제 버그 구문을 찾기까지 SW 실행을 역추적 하면서 찾기 때문에 많은 개발자 시간 및 노력이 필요하다. 따라서 오류가 발견된 후부터 수정할 때 까지 수 주의 기간이 드는 경우가 빈번하며 심지어 수 개월 또는 수 년 동안 오류를 수정하기 못하는 경우도 있다. 최근 제안된 SW 변형 기반 버그 탐색기법 (mutation-based fault localization)의 경우 관찰 기반 버그 탐색기법 (spectrum-based fault localization)보다 더 정확하게 버그 위치 탐색을 수행한다. SW 변형 기반 버그 탐색 기법은 다음 두 개의 관찰/가정에 기반하고 있다 (m_c는 정상적인 구문을 변형하여 생성한 변형 프로그램이고 m_f는 버그 구문을 변형하여 생성한 변형 프로그램이다).

1. 대상SW의 성공 테스트케이스를 m_c와 m_f에 실행하면, m_f 보다 m_c에서 테스트가 실패할 가능성이 더 높다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

2. 대상SW의 실패 테스트케이스를 m_c와 m_f에 실행하면, m_c 보다 m_f에서 테스트가 성공할 가능성이 더 높다.

따라서 분석 대상 프로그램과 변형 프로그램에서의 테스트 결과들을 비교하여 오류의 위치를 위치를 추정할 수 있다 (그림 5 참조).

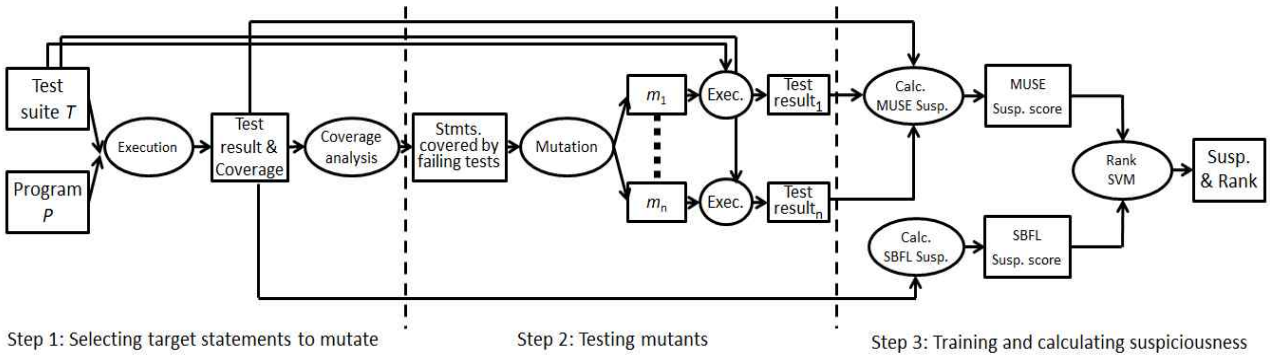


그림 6. 기계학습 기반 SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 자동 생성 기술 개요

그림 6은 본 연구에서 새로 개발한 기계학습 기반 SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 자동 생성 기술의 개요를 나타낸다. 기존 SW 변형 분석 기반 기법과 SW 관찰 분석 기반 기법을 결합하여 각 기법의 장점을 효과적으로 활용할 수 있다. 두 기법 사이의 중요도를 효과적으로 학습하여 활용하기 위해 RankSVM 기계학습 기법을 사용하였다. 새로 개발한 기계학습 기반 SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 자동 생성 기술은, 아래 알고리즘 3의 세 단계로 수행된다.

○ Step 1: 원본 프로그램의 분석 대상 구문 선택

입력으로 주어진 테스트 집합 T를 분석 대상 프로그램 P에 실행하여 분석 대상 구문을 확인한다. 분석 대상 구문은 버그 구문, 즉 테스트를 실패하게 만들 수 있는 가능성이 있는 구문이다. 따라서 최소한 개 이상의 실패한 테스트가 실행한 구문이 분석 대상 구문이며 실패한 테스트가 전혀 실행하지 않은 구문은 우리가 찾고자 하는 버그 구문과는 무관하기 때문에 분석 대상 구문에서 제외한다.

○ Step 2: 변형 프로그램 생성 및 테스트 수행

Step 1에서 찾은 분석 대상 구문에, MBFL의 정확도는 높이고 실행시간은 줄이는 변형 연산자를 적용하여 변형 프로그램을 생성하고 생성한 변형 프로그램에, 역시 MBFL의 정확도를 높이며 실행시간을 줄이는 테스트 집합 T를 실행하여 변형 프로그램 테스트를 수행한다. 이 때 분석 대상 프로그램 P에서는 성공했으나 변형 프로그램에서는 실패한 테스트(가설/관찰 1)와 분석 대상 프로그램 P에서는 실패했으나 변형 프로그램에서는 성공한 테스트(가설/관찰 2)의 정보를 추출한다.

○ Step 3: 변형 프로그램 테스트 결과 및 관찰 분석 결과를 활용하여 버그 의심도 순위 계산

Step 2에서 수행한 변형 프로그램 테스트 결과와 관찰 분석 결과를 활용하여 버그 의심도 순위를 계산한다.

버그 의심도는 각 분석 대상 구문이 실제 오류인지 아닌지를 나타내는 정량 지표이다. 버그 의심도의 상대적인 대소 비교를 통해 분석 대상 구문이 실제 오류일지 아닐지를 보여주는 의심도 순위를 메긴다. RankSVM 과 같은 기계학습 기반의 알고리즘 특성상 기계학습으로 계산한 버그 의심도의 절대적인 값의 차이는 크게 중요하지 않으며 버그 구문의 의심도 순위를 계산하기 위한 상대적인 대소 관계가 중요하다. 버그 의심도를 구체적으로 계산하기 위해서 SW 변형 분석 결과 7개와 SW 관찰 분석 결과 3개의 특징(feature)을 사용한 기계학습/추론을 수행하였다. 기계학습 방법으로는 순위를 계산하는데 효과적이라고 알려진 RankSVM 기법을 활용하였다.

알고리즘 3. SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 자동 생성 기술

나) 연구 기술 적용 실험 결과 (세부 목표 증빙 1번 참고)

개발한 SW 디버깅 정보 자동 생성 기술이 실제 효과적으로 버그 구문의 위치를 추정할 수 있는지 확인하기 위해 4개 C 오픈 소스 프로그램(Coreutils, find, grep, make)의 65개 버그를 대상으로 실험을 수행하였다. 각 타겟 프로그램의 크기는 평균적으로 36.5 천 줄이며 평균 1.1개의 실패한 테스트(버그를 재현하는 테스트)와 273.8개의 성공한 테스트(버그를 재현하지 않는 테스트)가 있었다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

연구질문 1: 기존에 효과적이라고 알려진 관찰 분석 기반 SW 디버깅 정보 생성 기술 Op2와 SW 변형 분석 기반 SW 디버깅 정보 생성 기술 MUSE와 비교해서 얼마나 더 정확하게 버그 위치를 추정할 수 있는가?
연구질문 2: SW 변형 프로그램을 임의로 샘플링하여 더 적은 수를 사용했을 때 버그 위치 추정 정확도는 얼마나 낮아지는가?

표 4 기계학습 기반 SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 생성 적용 결과

타겟 프로그램	버그 구문의 의심도 순위			버그 구문의 의심도 순위/전체 분석 대상 구문 수		
	Op2	MUSE	제안 기법	Op2	MUSE	제안 기법
Coreutils	163.79	57.00	47.37	30.08	10.34	8.53
find	383.29	66.86	50.14	39.93	7.23	5.43
grep	492.79	38.86	28.86	40.06	3.73	2.77
make	385.00	155.22	93.00	29.90	9.15	5.85
평균	343.18	82.42	56.62	34.30	7.92	5.88

표 4는 SW 변형 기반 자동 테스트 생성 기술을 적용했을 때의 버그 구문 의심도 순위의 절대 값 및 비율을 나타낸다. **제안하는 기법을 사용할 경우 분석 대상 구문의 평균 5.88%의 구문**만 분석하면 버그 구문을 찾을 수 있지만 **관찰 기반 기법(Op2)만 사용할 경우 제안 기법의 5.8배(34.30% 구문), 변형 분석 기반 기법(MUSE)만 사용할 경우 제안 기법의 1.3배(7.92%)를 분석**해야 한다.

표 5 기계학습 기반 SW 변형 분석 및 관찰 분석을 결합한 SW 디버깅 정보 생성 변형 프로그램 임의 선택 결과

타겟 프로그램	변형 프로그램 임의 선택 비율 (%)	버그 구문의 의심도 순위/전체 분석 대상 구문 수
Coreutils	10	16.77
	40	11.14
	70	9.91
	100	8.53
find	10	10.24
	40	6.85
	70	6.07
	100	5.43
grep	10	5.06
	40	3.61
	70	3.08
	100	2.77
make	10	10.67
	40	8.29
	70	7.14
	100	5.85
평균	10	10.33
	40	7.94
	70	6.98
	100	5.88

표 5는 변형 프로그램을 임의로 선택하여 더 적은 수를 사용할 경우 버그 구문 의심도 정확도가 얼마나 낮아지는지 나타낸다. 전체 변형 프로그램의 10%만 사용할 경우 실행 속도는 **전체 변형 프로그램을 분**

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

석할 때 보다 10배 빨라졌으며 정확도는 약 2배 정도 저하되었다(버그 구문을 찾기 위해 전체 분석 대상 구문의 10.33%를 분석). 전체 변형 프로그램의 70%만 사용할 경우 6.98% (100% 사용 대비 1.2배)를 분석해야 했으며, 전체 변형 프로그램의 40%만 사용하더라도 7.94%의 구문만 분석하면 되었다. 이는 기존 변형 SW 분석 기법인 MUSE의 7.92% 분석과 거의 같은 정확도를 나타낸다.

3. 연구개발결과의 중요성

효과적인 SW 테스팅 및 디버깅 연구는 SW 공학의 기반 기술로서 SW 품질 향상 다방면에 활용될 수 있다. 본 연구는 SW 품질 향상을 위해 표준적으로 사용되는 SW 테스팅 및 디버깅 과정을 자동화함으로써, 테스팅 및 디버깅의 병목이었던 개발자의 시간 부족 문제를 해결하고, 수만~수십만LOC 규모의 현실적인 SW에 적용 가능한 수준으로 개발하여 관련 연구자 뿐 아니라 SW 개발 현장의 개발자 및 테스트 전문가가 활용할 수 있는 수준의 기술을 개발하였다.

(1) 학문적 활용

- **유닛 테스팅 분야에 새로운 연구 문제(research problem) 제시**
  기존 SW 테스팅 자동화 연구는 테스트 입력 값 자동 생성에 집중하고 있으나, 본 연구는 테스트 입력 값 뿐 아니라 테스트 드라이버/스텁 함수 생성 자동화를 포함하여 기존 SW 테스팅 자동화 기법보다 버그 잡는 효과 및 테스트 커버리지를 효과적으로 높일 수 있었다.

- **동적 심볼릭 테스팅 분야에 테스팅 정확도를 향상하는 새로운 방안 제시**
  본 연구는 테스트 대상 유닛의 실제 실행 시나리오를 고려하여 SW 유닛 환경을 자동으로 생성함으로써 실제 실행 시나리오를 고려하지 않은 기존의 SW 유닛 환경 자동 생성 기법보다 더 정확한(거짓 경보 비율이 더 적은) 테스트 결과를 산출하였다.

- **SW변형 기술의 새로운 응용분야 제시**
  테스트 품질 평가와 회귀 테스트에 제한적으로 사용되던 SW 변형 기법을 사용하여 SW유닛 변형 테스트를 생성함으로써, SW 테스팅의 효과성을 높이고 SW 변형 기법 활용의 새 지평을 열었다.

(2) 기술적/산업적 활용

- **SW 품질, 생산성, 경쟁력 강화**
  본 연구는 SW 테스팅 및 디버깅 과정을 자동화함으로써 SW 개발 비용 중 가장 많은 비율을 차지하는 SW 품질관리 및 유지보수에 비용을 줄이고 SW 융복합산업의 경쟁력을 강화시켰다. 특히, 현대모비스와의 산학협력을 통해, 본 연구 결과를 적용했을 때 차량제어 SW의 테스팅 비용이 50~70% 절감됨을 실증적으로 입증하였다.

- **실제 SW 산업의 필요에 부응하는 연구 개발**
  본 연구는 SW 코드를 분석하여 SW품질을 향상시키는 연구이므로, SW융복합 산업계가 당면하고 있는 SW 품질 문제에 해결책이 된다.

- **오픈소스 SW도구 개발을 통한 연구 결과 파급**
  본 연구의 성과로 개발한 SW도구를 오픈소스 프로젝트 형태로 공개하여 다양한 산업체, 연구소에서 활용 가능하도록 함으로써, 연구 결과를 사회 전반에 효과적으로 보급되도록 지원하였다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

4. 참고문헌

[1] K. Sen, D. Marinov, G. Agha, CUTE: A concolic unit testing engine for C, in: European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE), 2005.

[2] S. Artzi, A. Kiezun, J. Dolby, F. Tip, D. Dig, A. Paradkar, M. Ernst, Finding bugs in dynamic web applications, in: International Symposium on Software Testing and Analysis (ISSTA), 2008.

[3] C. Cadar, D. Dunbar, D. Engler, KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs, in: Operating System Design and Implementation (OSDI), 2008.

[4] D. Molnar, X. C. Li, D. A. Wagner, Dynamic test generation to find integer bugs in x86 binary linux programs, in: USENIX Security Symposium, 2009.

[5] R. Sasnauskas, O. Landsiedel, M. h. Alizai, C. Weise, S. Kowalewski, K. Wehrle, KleeNet: Discovering insidious interaction bugs in wireless sensor networks before deployment, in: International Conference on Information Processing in Sensor Networks (IPSN), 2010.

[6] Y. Kim, M. Kim, Y. Kim, Y. Jang, Industrial application of concolic testing approach: A case study on libexif by using CREST-BV and KLEE, in: International Conference on Software Engineering (ICSE), 2012, SEIP track.

[7] T. Avgerinos, A. Rebert, S. Cha, D. Brumley, Enhancing symbolic execution with veritesting, in: International Conference on Software Engineering (ICSE), 2014.

[8] R. Banabic, G. Candea, R. Guerraoui, Finding trojan message vulnerabilities in distributed systems, in: International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2014.

[9] W. Visser, C. S. Pasareanu, S. Khurshid, Test input generation with Java PathFinder, in: International Symposium on Software Testing and Analysis (ISSTA), 2004.

[10] P. Godefroid, M. Y. Levin, D. Molnar, Automated whitebox fuzz testing, in: Network and Distributed System Security Symposium (NDSS), 2008.

[11] M. Kim, Y. Kim, Y. Jang, Industrial application of concolic testing on embedded software: Case studies, in: International Conference on Software Testing, Verification and Validation (ICST), 2012.

[12] E. Bounimova, P. Godefroid, D. Molnar, Billions and billions of constraints: Whitebox fuzz testing in production, in: International Conference on Software Engineering (ICSE), 2013.

[13] Y. Park, S. Hong, M. Kim, D. Lee, J. Cho, Systematic testing of reactive software with non-deterministic events: A case study on lg electric oven, in: ICSE Software Engineering In Practice, 2015.

[14] C. Csallner, Y. Smaragdakis, Jcrasher: an automatic robustness tester for java, Software-Practice and Experience (SPE) 34 (11) (2004) 1025{1050.

[15] G. Fraser, A. Arcuri, 1600 faults in 100 projects: Automatically finding faults while achieving high coverage with evosuite, Empirical Software Engineering (2013) 1-29.

[16] Y. Kim, Y. Kim, T. Kim, G. Lee, Y. Jang, M. Kim, Automated unit testing 835 of large industrial embedded software using concolic testing, in: Automated Software Engineering (ASE), 2013.

[17] D. Ramos, D. Engler, Under-constrained symbolic execution: Correctness checking for real code, in: USENIX Security Symposium, 2015.

[18] H. Do, S. Elbaum, G. Rothermel, Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact., Empirical Software Engineering Journal 10 (4) (2005) 405-435.

[19] The spec cpu 2006 benchmark suite, https://www.spec.org/cpu2006/.

[20] H. Cramer, Mathematical Methods of Statistics, Princeton University Press, 1946.

[21] K. Pearson, Notes on regression and inheritance in the case of two parents, in: Proceedings of the Royal Society of London, 1895.

[22] R. Rodrigues, V. Campos, F. Pereira, A fast and low-overhead technique to secure programs againtst integer overflows, in: CGO, 2013.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[23] M.D.Ernst, J.H.Perkins, P.J.Guo, S.McCamant, C.Pacheco, M.S.Tschantz, C.Xiao, The daikon system for dynamic detection of likely invariants, Science of Computer Programming 69 (1) (2007) 35-45.
[24] L. Moura, N. Bjorner, Z3: An ecient SMT solver, in: TACAS, 2008.
[25] C. Lattner, V. Adve, LLVM: A compilation framework for lifelong program analysis & transformation, in: CGO, 2004.
[26] J. Evans, Straightforward Statistics for the Behavioral Sciences, Brooks/Cole Publishing Company, 1996.
[27] P. Godefroid, N. Klarlund, K. Sen, DART: Directed automated random testing, in: Programming Language Design and Implementation (PLDI), 2005.
[28] N. Tillmann, J. Halleux, Pex - white box test generation for .net, in: Tests And Proofs (TAP), 2008.
[29] C. Pasareanu, P. Mehlitz, D. Bushnell, K. Gundy-burlet, M. Lowry, S. Person, M. Pape, Combining unit-level symbolic execution and system-level concrete execution for testing nasa software, in: ISSTA, 2008.
[30] A. Chakrabarti, P. Godefroid, Software partitioning for eective automated unit testing, in: International Conference on Embedded Software, 2006.
[31] S. Khurshid, C. Pasareanu, W. Visser, Generalized symbolic execution for model checking and testing, in: Tools and Algorithms for the Construction and Analysis of Systems, 2003.
[32] C. Pacheco, S. Lahiri, M. Ernst, T. Ball, Feedback-directed random test generation, in: International Conference on Software Engineering (ICSE), 2007.
[33] P.Garg, F.Ivancic, G.Balakrishnan, N.Maeda, A.Gupta, Feedback-directed unit test generation for C/C++ using concolic execution, in: International Conference on Software Engineering (ICSE), 2013.
[34] G. Fraser, A. Arcuri, Evosuite: automatic test suite generation for object-oriented software, in: Foundations of Software Engineering (FSE), 2011.
[35] L. Baresi, P. Lanzi, M. Miraz, TestFul: an evolutionary test approach for java, in: International Conference on Software Testing, Verification and Validation (ICST), 2010.
[36] S. Elbaum, H. Chin, M. Dwyer, M. Jorde, Carving and replaying differential unit test cases from system test cases, IEEE Transactions on Software Engineering (TSE) 35 (1) (2009) 29-45.
[37] H. Jaygarl, S. Kim, T. Xie, C. Chang, OCAT: Object capture based automated testing, in: International Symposium on Software Testing and Analysis (ISSTA), 2010.
[38] B. Pasternak, S. Tyszberowicz, A. Yehudai, GenUTest: A unit test and mock aspect generation tool, Software Tools for Technology Transfer 11 (4) (2009) 273-290.
[39] M. Islam, C. Csallner, Dsc+mosk: A test case + mock class generator in support of coding against interfaces, in: WODA, 2010.
[40] S. Galler, A. Maller, F. Wotawa, Automatically extracting mock object behavior from design by contract specification for test data generation, in: AST, 2010.
[41] D. Sa, M. Ernst, Mock object creation for test factoring, in: PASTE, 2004.
[42] D. Sa, S. artzi, J. Perkins, M. Ernst, Automatic test factoring for java, in: Automated Software Engineering (ASE), 2005.
[43] S. Moon, Y.Kim, M.Kim, S.Yoo, Ask the Mutants: Mutating Faulty Programs for Fault Localization, IEEE International Conference on Software Testing, Verification and Validation (ICST), Pages: 153 - 162, March 31-April 4, 2014

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

5. 연구성과

과학기술/학술적 연구성과(단위 : 건)													
전문학술지 논문게재				초청 강연 실적	학술대회 논문발표		지식재산권				수상 실적	출판실적	
국내논문		국외논문			국내	국제	출원		등록			저역서	보고서
SCI	비SCI	SCI	비SCI				국내	국외	국내	국외			
0	4	1	0	1	5	4	0	0	0	0	8	0	0

인력양성 및 연구시설(단위 : 명,건)							
학위배출		국내외 연수지원				산학강좌	연구기자재
박사	석사	장기		단기			
		국내	국외	국내	국외		
1	2	0	0	0	0	4	0

국제협력(단위 :명,건)					
과학자교류		국제협력기반			학술회의개최
국내과학자 해외파견	외국과학자 국내유치	MOU체결	국제공동연구	국제사업참여	학술회의개최(국내,국제 통합)
0	0	0	0	0	0

산업지원 및 연구성과 활용(단위 : 건)								
기술확산				연구성과활용(사업화 및 후속연구과제 등)				
기술실시계약	기술이전	기술지도	기술평가	후속연구추진	사업화추진중	사업화완료	기타목적활용	기술마케팅
0	0	0	0	0	0	0	0	0

기타 성과(단위 : 건)					
언론보도 성과	생명자원/신품종/화합물	원자력연구개발사업실적(원자력연구개발사업에한함)			
		기술보고서	설계문서	장비구축 및 개발	분석방법개발
2	0	0	0	0	0

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

전문학술지 논문게재 성과정보

과제번호	게재연월	논문제목	총저자명	출처	학술지명	권(호)	학술지구분	sci여부	impact Factor	국제공동 연구논문	기여도
2016R1A2B4008113	201702	MUSEUM: Debugging real-world multilingual programs using mutation analysis	Hong, Shin; Kwak, Taehoon; Lee, Byeongcheol; Jeon, Yiru; Ko, Bongseok; Kim, Yunho; Kim, Moonzoo;	SCI	INFORMATION AND SOFTWARE TECHNOLOGY	82(0)	국외	SCI등재	2.694	아니오	20
2016R1A2B4008113	201702	안드로이드 커널 모듈 취약점 탐지를 위한 자동화된 유닛 테스트 생성 기법	김윤호;김문주	KCI	정보과학회논문지	44(2)	국내	SCI미등재		아니오	85
2016R1A2B4008113	201804	효과적인 변이 분석을 위한 C 프로그램 변이 도구 비교: Proteum과 Milu를 사용한 사례 연구	김윤호;김현우;양웅규;김문주	KCI	정보과학회논문지	45(4)	국내	SCI미등재		아니오	100
2016R1A2B4008113	201810	C 프로그램의 동적 및 정적 분석을 통한 시스템 실행에서의 유닛 입력 값 자동 수집 및 재연	임현수;김윤호;김문주	KCI	정보과학회논문지	45(10)	국내	SCI미등재		아니오	30
2016R1A2B4008113	201810	Concolic 테스팅 도구 CROWN의 적용 가능성 및 사용성 개선 연구	김현우;김윤호;김문주	KCI	정보과학회논문지	45(10)	국내	SCI미등재		아니오	30

국제학술대회 초청강연실적 성과정보

과제번호	발표연월	학술대회명	발표자	강연주제	개최국
2016R1A2B4008113	201810	Formal Aspects of Component Software	김문주	Lessons Learned from Automated Analysis of Industrial SW for 15 Year	대한민국

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

학술대회 논문발표 성과정보

과제번호	발표년월	학술대회명	저자	논문제목	학술대회구분	개최국
2016R1A2B4008113	201612	한국정보과학회 2016년 동계학술대회	김윤호;김현우;양웅규;김문주	효과적인 변이 분석을 위한 C 프로그램 변이 도구 비교	국내학술대회	대한민국
2016R1A2B4008113	201712	2017 한국컴퓨터종합학술대회	임현수; 김윤호; 김문주	시스템 테스트 케이스를 이용한 C 프로그램의 동적 유닛 입력 값 자동 수집 및 재연 기술	국내학술대회	대한민국
2016R1A2B4008113	201712	2017 한국컴퓨터종합학술대회	Phan Duy Loc; 고봉석; 김윤호; 김문주	COMUT: 사용자의 의도대로 효과적인 변이를 생성할 수 있는 C 프로그램 변이 도구	국내학술대회	대한민국
2016R1A2B4008113	201801	한국 소프트웨어공학 학술대회	김현우;김윤호,김문주	Concolic 테스팅 도구 CREST 의 사용자 친화성 향상 연구: Windows OS 로의 포팅과 개선된 CREST UI 을 통한 CREST 활용 및 분기 커버리지 분석 작업의 효율 증가	국내학술대회	대한민국
2016R1A2B4008113	201812	한국소프트웨어종합학술대회	박건우;송형곤;이주현;조규태;김윤호;김문주	국방 무기 체계 SW 품질 향상을 위한 Concolic 테스팅 기술	국내학술대회	대한민국
2016R1A2B4008113	201804	International Conference on Software Testing(ICST)	김윤호;홍신;고봉석;Phan Duy Loc;김문주	Invasive Software Testing: Mutating Target Programs to Diversify Test Exploration for High Test Coverage	국제학술대회	스웨덴
2016R1A2B4008113	201804	International Conference on Software Testing, Verification and Validation Workshops (ICSTW)	Phan Duy Loc;김윤호;김문주	MUSIC: Mutation Analysis Tool with High Configurability and Extensibility	국제학술대회	스웨덴
2016R1A2B4008113	201805	International Conference on Software Engineering (ICSE)	김윤호;최윤자;김문주	Precise Concolic Unit Testing of C Programs Using Extended Units and Symbolic Alarm Filtering	국제학술대회	스웨덴
2016R1A2B4008113	201905	International Conference on Software Engineering	Yunho Kim;Dongju Lee; Junki Baek; Moonzoo Kim	Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry	국제학술대회	캐나다

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

수상실적 성과정보

과제번호	수상연월	수상명	수상자		시상기관	시상국가	수상사유
			성명	소속기관			
2016R1A2B4008113	201612	한국정보과학회 2016년 동계학술대회 우수논문상	김윤호;김현우;양웅규;김문주	한국과학기술원	한국정보과학회	대한민국	한국정보과학회 2016년 동계학술대회 우수논문상 논문제목: 효과적인 변이 분석을 위한 C 프로그램 변이 도구 비교: Proteum과 Milu를 사용한 사례 연구
2016R1A2B4008113	201702	삼성 휴먼테크 동상	김윤호	한국과학기술원	삼성전자(주)	대한민국	삼성전자 주최 휴먼테크 논문대상 동상 논문제목: Effective Concolic Unit Testing based on Realistic Unit
2016R1A2B4008113	201702	KAIST 전산학부 우수 박사학위논문상	김윤호	한국과학기술원	한국과학기술원	대한민국	2017년 졸업생 대상 우수 박사학위 논문상 논문제목: Automated Unit Test Generation with Realistic Unit Context Synthesis for Low False Alarms
2016R1A2B4008113	201712	2017 한국컴퓨터종합학술대회 우수논문상	Phan Duy Loc; 고봉석; 김윤호; 김문주	한국과학기술원	한국정보과학회	대한민국	2017 한국컴퓨터종합학술대회 우수논문상 논문제목: COMUT: 사용자의 의도대로 효과적인 변이를 생성할 수 있는 C 프로그램 변이 도구
2016R1A2B4008113	201712	2017 한국컴퓨터종합학술대회 우수논문상	임현수; 김윤호; 김문주	한국과학기술원	한국정보과학회	대한민국	2017 한국컴퓨터종합학술대회 우수논문상 논문제목: 시스템 테스트 케이스를 이용한 C 프로그램의 동적 유닛 입력 값 자동 수집 및 재연 기술

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

수상실적 성과정보

과제번호	수상연월	수상명	수상자		시상기관	시상국가	수상사유
			성명	소속기관			
2016R1A2B4008113	201801	2018 한국 소프트웨어공학 학술대회 우수논문상	김현우;김윤호;김문주	한국과학기술원	한국정보과학회	대한민국	한국소프트웨어공학학술대회 2018에서 우수한 연구 성과를 인정받아 우수논문상 수상 논문제목 : Concolic 테스팅 도구 CREST 의 사용자 친화성 향상 연구: Windows OS 로의 포팅과 개선된 CREST UI 을 통한 CREST 활용 및 분기 커버리지 분석 작업의 효율 증가
2016R1A2B4008113	201804	11th International Conference on Software Testing, Verification and Validation (ICST) Distinguished Paper Award	김윤호;홍신;고봉석;Phan Duy Loc;김문주	한국과학기술원	IEEE International Conference on Software Testing, Verification and Validation (ICST)	스웨덴	IEEE International Conference on Software Testing, Verification and Validation (ICST, 정보과학회에서 SE 분야 A급 우수학술대회로 인정)) 2018 국제학술대회에서 뛰어난 연구 성과를 인정받아 Distinguished paper award 수상 논문제목 : Invasive Software Testing: Mutating Target Programs to Diversify Test Exploration for High Test Coverage
2016R1A2B4008113	201812	2018 한국소프트웨어 종합학술대회 최우수논문상	박건우;송형곤;이주현;조규태;김윤호;김문주	한국과학기술원	한국정보과학회	대한민국	2018 한국소프트웨어 종합학술대회에서 뛰어난 연구 성과를 인정받아 최우수논문상 수상 논문제목 : 국방 무기 체계 SW 품질 향상을 위한 Concolic 테스팅 기술

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

학위배출인력 성과정보							
과제번호	학위취득연월	학위구분	학위취득자				진로
			성명	성별	대학	학과	
2016R1A2B4008113	201702	박사	김윤호	남성	한국과학기술원	전산학부	취업(박사후연구원 포함)
2016R1A2B4008113	201808	석사	김현우	남성	한국과학기술원	전산학부	취업(박사후연구원 포함)
2016R1A2B4008113	201902	석사	임현수	남성	한국과학기술원	전산학부	취업(박사후연구원 포함)

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

관련강좌 프로그램 개설 성과정보						
과제번호	개설기간	프로그램명	프로그램 내용	교육대상	교육시간	인원
2016R1A2B4008113	20160905~20161219	Automated Software Testing	소프트웨어 테스팅 및 분석 자동화를 위한 다음 내용들을 교육한다. Part I: Program coverage criteria and program analysis Part II: Concolic testing (a.k.a., dynamic symbolic execution) Part III: Model checking	KAIST 학부생 및 대학원생	48	29
2016R1A2B4008113	20170829~20171212	Automated Software Analysis	This class covers automated SW testing/verification techniques to detect SW bugs effectively and efficiently. In particular, this class focuses to teach techniques that automatically generate test cases to achieve high code coverage and, thus, to detect many bugs. The class aims to teach practical applications of advanced testing/verification techniques as well as their underlying algorithms. This class guides students to use various open-source software testing/verification tools through HWs and learn the underlying mechanisms to maximize the performance of automated testing/verification. Part I: Program coverage criteria and program analysis Part II: Model checking and Test Oracles Part III: Concolic testing (a.k.a., dynamic symbolic execution) Part IV: Verification engince - SAT/SMT solver	KAIST 학부생 및 대학원생	48	11
2016R1A2B4008113	20171113~20171113	Coverage criteria for white-box testing	현대모비스의 SW테스팅 실무자 20여명을 대상으로, 커버리지 기반 white-box 테스팅의 기반 기술들을 체계적으로 강의 - graph-based software testing coverage - logic-based software testing coverage - mutation-based software testing coverage	현대모비스 SW테스팅 실무자	8	20

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

관련강좌 프로그램 개설 성과정보						
과제번호	개설기간	프로그램명	프로그램 내용	교육대상	교육시간	인원
2016R1A2B4008113	20180827~20181212	Automated Software Analysis	This class covers automated SW testing/verification techniques to detect SW bugs effectively and efficiently. In particular, this class focuses to teach techniques that automatically generate test cases to achieve high code coverage and, thus, to detect many bugs. The class aims to teach practical applications of advanced testing/verification techniques as well as their underlying algorithms. This class guides students to use various open-source software testing/verification tools through HWs and learn the underlying mechanisms to maximize the performance of automated testing/verification. Part I: Program coverage criteria and program analysis Part II: Model checking and Test Oracles Part III: Concolic testing (a.k.a., dynamic symbolic execution) Part IV: Verification engine - SAT/SMT solver	KAIST 학부생 및 대학원생	48	11

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

언론보도성과			
과제번호	2016R1A2B4008113		
보도게재일자	20180722	보도및게재처	연합뉴스
기사제목	현대모비스, AI 기반 소프트웨어 검증시스템 도입…"효율 2배로"		
관련연구업적	Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry		

과제번호	2016R1A2B4008113		
보도게재일자	20180722	보도및게재처	조선비즈
기사제목	현대모비스, AI 기반 소프트웨어 검증시스템 도입...미래차 기술 활용		
관련연구업적	Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry		

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

<주관연구책임자(공동연구원 포함) 대표적 연구실적>

○ 논문 및 특허 실적(최대 5개 작성)

번호	구분(논문/특허)	논문명/특허명	소속기관명	역할	논문게재지/특허등록국가	논문게재일/특허등록일	특기사항(I.F. 등)
1	논문	Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry	한국과학기술원	교신저자	Intl. Conf. on Software Engineering (ICSE)	2019.05.28	정보과학회 최우수학술대회
2	논문	Precise concolic unit testing of C programs using extended units and symbolic alarm filtering	한국과학기술원	교신저자	Intl. Conf. on Software Engineering (ICSE)	2018.05.31	정보과학회 최우수학술대회
3	논문	Invasive Software Testing: Mutating Target Programs to Diversify Test Exploration for High Test Coverage	한국과학기술원	교신저자	IEEE International Conference on Software Testing, Verification and Validation (ICST)	2018.04.11	정보과학회 우수학술대회 우수논문상 수상
4	논문	MUSEUM:Debugging real-world multilingual programs using mutation analysis	한국과학기술원	공동저자	Information and Software Technology (IST)	2017.02.01	2.627
5	논문	MUSIC: Mutation Analysis Tool with High Configurability and Extensibility	한국과학기술원	교신저자	Mutation Workshop	2018.04.10	

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

[붙임1]

〈 대 표 연 구 실 적 요 약 문 〉

대표연구실적 요약문			
연구업적 제목 (논문명/특허명)	Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry		
게재지	Intl. Conf. on Software Engineering (ICSE)		
게재연도/등록(출원)연도	2019		
주관연구책임자 또는 참여연구원 성명	김문주	참여자수	4
SCI 등재 여부	해당없음	IF 및 피인용횟수 (SCI)	0 (Google scholar)
SCOPUS 등재 여부	해당없음	피인용횟수 (SCOPUS)	해당없음

요 약 문

▣ 초록(abstract) 또는 요약문

The importance of automotive software has been rapidly increasing because software now controls many components in motor vehicles such as window controller, smart-key system, and tire pressure monitoring system. Consequently, the automotive industry spends a large amount of human effort testing automotive software and is interested in automated software testing techniques that can ensure high-quality automotive software with reduced human effort.

In this paper, we report our industrial experience applying concolic testing to automotive software developed by Hyundai Mobis. We have developed an automated testing framework MAIST that automatically generates the test driver, stubs, and test inputs to a target task by applying concolic testing. As a result, MAIST has achieved 90.5% branch coverage and 77.8% MC/DC coverage on the integrated body unit (IBU) software. Furthermore, it reduced the cost of IBU coverage testing by reducing the manual testing effort for coverage testing by 53.3%.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대표적 연구실적 사본 〉

2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)

Concolic Testing for High Test Coverage and Reduced Human Effort in Automotive Industry

Yunho Kim
School of Computing
KAIST
Daejeon, South Korea
yunho.kim03@gmail.com

Dongju Lee
Software Verification Team
Hyundai Mobis
Yongin, South Korea
dongju.lee@mobis.co.kr

Junki Baek
Software Verification Team
Hyundai Mobis
Yongin, South Korea
jk.baek@mobis.co.kr

Moonzoo Kim
School of Computing
KAIST
Daejeon, South Korea
moonzoo@cs.kaist.ac.kr

***Abstract*—The importance of automotive software has been rapidly increasing because software now controls many components in motor vehicles such as window controller, smart-key system, and tire pressure monitoring system. Consequently, the automotive industry spends a large amount of human effort testing automotive software and is interested in automated software testing techniques that can ensure high-quality automotive software with reduced human effort.**

In this paper, we report our industrial experience applying concolic testing to automotive software developed by Hyundai Mobis. We have developed an automated testing framework MAIST that automatically generates the test driver, stubs, and test inputs to a target task by applying concolic testing. As a result, MAIST has achieved 90.5% branch coverage and 77.8% MC/DC coverage on the integrated body unit (IBU) software. Furthermore, it reduced the cost of IBU coverage testing by reducing the manual testing effort for coverage testing by 53.3%.

***Keywords*-Automated test generation, concolic testing, automotive software, coverage testing**

I. Introduction

The automotive industry has developed automotive software to control various components in the motor vehicle, for example, the body control module (BCM), smart-key system (SMK), and tire pressure monitoring system (TPMS) [1], [2]. As automotive software becomes larger and more complex with the addition of newly introduced automated features (e.g., advanced driver assistance systems) and more sophisticated functionality (e.g., driving mode systems) [3], [4], the cost of testing automotive software is rapidly increasing. Also, it is difficult for human engineers to develop test inputs that can ensure high-quality automotive software within tight software development schedules and budgets. To resolve these problems, the automotive industry is trying to apply automated software testing techniques to achieve high code coverage with reduced human effort.

Concolic testing [5] (a.k.a. dynamic symbolic execution [6]) has been applied to automatically generate test inputs for software in various industries. Concolic testing combines dynamic concrete execution and static symbolic execution to explore all possible execution paths of a target program, which can achieve high code coverage. Concolic testing has been applied to various industrial projects (e.g., flash memory device driver [7], mobile phone software [8], [9], and large-scale embedded software [10]) and has effectively improved the quality of industrial software by increasing test coverage and detecting corner-case bugs with modest human effort.

While we were working to apply concolic testing to automotive software developed by Mobis, we observed the following technical challenges to resolve to successfully apply automated test generation techniques:

1) We need to generate test drivers and stubs carefully to achieve high unit test coverage while avoiding generating test cases corresponding to the executions that are not feasible at the system-level. Otherwise (e.g., generating naive test drivers and stubs that provide unconstrained symbolic inputs to every function in a target program), we will waste human effort to manually filter out infeasible tests that lead to misleading high coverage and false alarms.
2) Current concolic testing tools do not support symbolic bit-fields in C which are frequently used for automotive software.[1] For example, automotive software uses bit-fields in message packets in the controller area network (CAN) bus to save memory and bus bandwidth. However, most concolic testing tools do not support symbolic bit-fields since a bit-field does not have a memory address (Sect. III-D) and most programs running on PCs rarely use bit-fields.
3) Although automotive software uses function pointers to simplify code to dynamically select a function to execute, concolic testing techniques and tools do not support symbolic function pointers due to the limitation of SMT (Satisfiability Modulo Theories) solvers.

To address the above challenges, we have developed an automated testing framework MAIST. MAIST automatically generates the test driver, stubs, and test inputs for a target unit using concolic testing. MAIST achieved 90.5% branch coverage and 77.8% MC/DC coverage of the IBU software (Sect. V-A). Also, MAIST reduced the manual testing effort by 53.3% for coverage testing of IBU (Sect. V-C). [2]

The main contributions of this paper are as follows:

[1]A bit-field `x:m` is an integer variable in a `struct` variable which has only `m` bits. For example, `unsigned int x:3` can represent only 0 to 7.
[2]Several newspapers reported these successful results [11]–[13].

978-1-7281-1760-7/19/$31.00 ©2019 IEEE
DOI 10.1109/ICSE-SEIP.2019.00024

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

VII. Related Work

A. Concolic Testing Techniques

1) Virtual machine (VM)-based Techniques: These techniques run as a layer on top of a VM to interpret compiled IR code of a target program p and obtain symbolic path formulas from p's executions. This approach can conveniently obtain all detailed run-time execution information of p available to a VM. However, test generation speed is slow due to slow IR interpretation and customizing the tools is non-trivial due to complex VM infrastructure. PEX [6] targets C# programs that are compiled to Microsoft .Net binaries (now available as IntelliTest [24] in Visual Studio). KLEE [19] (and its distributed version Cloud9 [25]) targets LLVM [16] binaries. jFuzz [26] and Symbolic PathFinder [27] target Java bytecode programs on top of Java PathFinder [28].

2) Instrumentation-based Techniques: These techniques insert probes in target source code to obtain dynamic execution information to build symbolic path formulas. This approach is lighter and easier-to-customize than the VM-based one. However, it requires complex source code parsing and instrumentation. CUTE [5], DART [29], CREST [22] (and its distributed version SCORE [30]), CROWN [14] target C programs and jCUTE [31] and CATG [32] target Java programs. MAIST uses CROWN as its concolic testing engine because CROWN (and its predecessor CREST) has been successfully applied to various industrial projects (Sect. VII-C).

B. Automated Test Driver/Stub Generation

DART [29] generates symbolic unit test drivers, but not symbolic stubs for concolic testing. To avoid the infeasible test generation issue, DART targets public API functions in libraries because such functions should accept all possible inputs. UC-KLEE [33] directly starts symbolic execution from a target function using lazy initialization [34] and calls all the functions directly or transitively invoked by the target function. Thus, DART and UC-KLEE target code of a function and its all callee functions, which can make concolic testing achieve low coverage within a fixed amount of testing time because the symbolic execution space can become very large. Chakrabarti and Godefroid [35] statically divide a static call graph into partitions using topological information and consider the partitions as testing targets for concolic testing. However, the proposed partitioning method does not consider semantic information on the relation between functions.

CONBOL [10], [36] generates symbolic unit test driver and stubs for functions in large-scale embedded software. It replaces all functions invoked by a target function by symbolic stubs. CONBOL uses target project specific false alarm reduction heuristics, which may not be effective for other projects. SmartUnit [37] generates symbolic test drivers and stubs for target C functions (the authors do not clearly describe how SmartUnit generates driver and stubs). The paper reports that SmartUnit achieved high coverage, but it does not report how many false alarms were raised. We could not directly compare the performance of MAIST with CONBOL and SmartUnit since they are not publicly available.

CONBRIO [14] constructs extended units as testing target units by using highly-relevant callee functions of a target function. The relevance between functions is computed based on system-level execution profiles. Targeting the extended units, CONBRIO achieves high bug detection power with low false alarm ratio. However, CONBRIO was not applicable to IBU in this testing project because we could not obtain IBU execution profiles by driving physical motor vehicles.

C. Industrial Application of Concolic Testing

Microsoft developed SAGE [38], [39] for x86/64 binaries to detect security vulnerabilities of Windows and Office products. Bardin and Herrmann [40], [41] developed and applied OSMOSE to embedded software. They translated machine code into an intermediate representation to apply concolic testing. Intel developed and applied MicroFormal [42] for Intel CPU's microcode. Fujitsu developed KLOVER [43] by extending KLEE targeting C++ programs. Zhang et al. developed and applied SmartUnit [37] to embedded software to achieve high branch and MC/DC coverage. Kim et al. applied CREST to the Samsung flash memory device driver code [7], [44]. They also compared CREST and KLEE for industrial use of concolic testing for the Samsung mobile phone software [9] and developed a systematic event-sequence generation framework using CREST for LG electric oven [45].

These industrial case studies focused on increasing the test effectiveness but did not report how much manual testing effort was saved. In contrast, this paper reports how much MAIST saved the manual testing effort (in man-months) in the automotive company (Sect. V-B). Also, we have shared the technical challenges and the solutions for the application of concolic testing to automotive software, which can promote field engineers to adopt concolic testing in their projects.

VIII. Conclusion

We have presented the industrial study of applying concolic testing to the automotive software developed by Mobis. After we identified and addressed the technical challenges of applying concolic testing to automotive software, we have developed an automated test generation framework MAIST. It generates a task-oriented test driver and stubs to reduce infeasible test executions and supports symbolic bit-fields and symbolic setting for function pointers that automotive software uses. MAIST has achieved 90.5% branch coverage and 77.8% MC/DC coverage on IBU and reduced the manual coverage testing effort by 53%. As future work, we plan to generate more precise test driver/stubs to reach uncovered branches. Also, we will apply MAIST to other automotive software to increase the economic benefit of MAIST.

Acknowledgement

We would like to thank Acheong Lee and Hyunwoo Kim for their initial effort of applying MAIST to IBU. This research has been supported by Hyundai Mobis, Next-Generation Information Computing Development Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (MSIT) (No. NRF-2017M3C4A7068177), Basic Science Research Program

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

through NRF funded by MSIT (NRF-2016R1A2B4008113), Basic Science Research Program through NRF funded by the Ministry of Education (NRF-2017R1D1A1B03035851), and MSIT, under the National Program for Excellence in SW (2016-0-00018), supervised by the IITP.

REFERENCES

[1] M. Broy, "Challenges in automotive software engineering," in *International Conference on Software Engineering (ICSE)*. New York, NY, USA: ACM, 2006, pp. 33–42.

[2] M. Broy, I. H. Kruger, A. Pretschner, and C. Salzmann, "Engineering automotive software," *Proceedings of the IEEE*, vol. 95, no. 2, pp. 356–373, Feb 2007.

[3] F. Falcini, G. Lami, and A. M. Costanza, "Deep learning in automotive software," *IEEE Software*, vol. 34, no. 3, pp. 56–63, May 2017.

[4] M. Traub, A. Maier, and K. L. Barbehön, "Future automotive architecture and the impact of it trends," *IEEE Software*, vol. 34, no. 3, pp. 27–32, May 2017.

[5] K. Sen, D. Marinov, and G. Agha, "CUTE: A concolic unit testing engine for C," in *European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE)*, 2005, pp. 263–272.

[6] N. Tillmann and W. Schulte, "Parameterized unit tests," in *European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE)*, 2005.

[7] M. Kim, Y. Kim, and Y. Choi, "Concolic testing of the multi-sector read operation for flash storage platform software," *Formal Aspects of Computing*, vol. 24, no. 3, pp. 355–374, May 2012.

[8] M. Kim, Y. Kim, and Y. Jang, "Industrial application of concolic testing on embedded software: Case studies," in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2012, pp. 390–399.

[9] Y. Kim, M. Kim, Y. Kim, and Y. Jang, "Industrial application of concolic testing approach: A case study on libexif by using CREST-BV and KLEE," in *International Conference on Software Engineering (ICSE)*, 2012, pp. 1143–1152.

[10] Y. Kim, Y. Kim, T. Kim, G. Lee, Y. Jang, and M. Kim, "Automated unit testing of large industrial embedded software using concolic testing," in *Automated Software Engineering (ASE)*, 2013, pp. 519–528.

[11] "Hyundai mobis increases ai use for improved efficiency," http://www.koreaherald.com/view.php?ud=20180722000186, accessed: 2018-10-10.

[12] "Hyundai mobis taps ai for car software," https://www.koreatimes.co.kr/www/tech/2018/07/419_252629.html, accessed: 2018-10-01.

[13] "Hyundai mobis introduces ai-based software verification system," http://www.businesskorea.co.kr/news/articleView.html?idxno=23830, accessed: 2018-10-01.

[14] Y. Kim, Y. Choi, and M. Kim, "Precise concolic unit testing of c programs using extended units and symbolic alarm filtering," in *International Conference on Software Engineering (ICSE)*, 2018, pp. 315–326.

[15] "Open hub, the open source network," https://www.openhub.net/.

[16] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-directed and Runtime Optimization*, ser. CGO '04, 2004.

[17] G. Necula, S. McPeak, S. Rahul, and W. Weimer, "CIL: Intermediate language and tools for analysis and transformation of c programs," in *Compiler Construction (CC)*, 2002.

[18] J. Burnim, "CREST - automatic test generation tool for C," http://code.google.com/p/crest/.

[19] C. Cadar, D. Dunbar, and D. Engler, "KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs," in *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'08, 2008, pp. 209–224.

[20] N. Williams, B. Marre, P. Mouy, and M. Roger, "Pathcrawler: automatic generation of path tests by combining static and dynamic analysis," in *EDCC*, 2005.

[21] R. Baldoni, E. Coppa, D. C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 50:1–50:39, May 2018.

[22] J. Burnim and K. Sen, "Heuristics for scalable dynamic test generation," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2008-123, Sep 2008.

[23] "Testwell CTC++," https://www.verifysoft.com/en_ctcpp.html, accessed: 2018-10-01.

[24] N. Tillmann and J. De Halleux, "Pex: White box test generation for .NET," in *Proceedings of the 2Nd International Conference on Tests and Proofs*, ser. TAP'08. Berlin, Heidelberg: Springer-Verlag, 2008, pp. 134–153.

[25] L. Ciortea, C. Zamfir, S. Bucur, V. Chipounov, and G. Candea, "Cloud9: a software testing service," *ACM SIGOPS Operating Systems Review*, vol. 43, no. 4, pp. 5–10, 2010.

[26] K. Jayaraman, D. Harvison, V. Ganesh, and A. Kiezun, "jFuzz: A concolic whitebox fuzzer for Java," in *NFM*, 2009.

[27] C. S. Păsăreanu, P. C. Mehlitz, D. H. Bushnell, K. Gundy-Burlet, M. Lowry, S. Person, and M. Pape, "Combining unit-level symbolic execution and system-level concrete execution for testing NASA software," in *Proceedings of the 2008 International Symposium on Software Testing and Analysis*, 2008, pp. 15–26.

[28] W. Visser, K. Havelund, G. Brat, and S. Park, "Model checking programs," in *ASE*, Sep. 2000.

[29] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2005, pp. 213–223.

[30] M. Kim, Y. Kim, and G. Rothermel, "A scalable distributed concolic testing approach: An empirical evaluation," in *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, April 2012, pp. 340–349.

[31] K. Sen and G. Agha, "CUTE and jCUTE : Concolic unit testing and explicit path model-checking tools," in *CAV*, 2006.

[32] "Catg: Concolic testing engine for java," https://github.com/ksen007/janala2, accessed: 2018-10-01.

[33] D. A. Ramos and D. Engler, "Under-constrained symbolic execution: Correctness checking for real code," in *Proceedings of the 24th USENIX Conference on Security Symposium*, ser. SEC'15, 2015, pp. 49–64.

[34] S. Khurshid, C. S. Păsăreanu, and W. Visser, "Generalized symbolic execution for model checking and testing," in *Proceedings of the 9th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, ser. TACAS'03, 2003, pp. 553–568.

[35] A. Chakrabarti and P. Godefroid, "Software partitioning for effective automated unit testing," in *Proceedings of the 6th International Conference on Embedded Software*, ser. EMSOFT '06. New York, NY, USA: ACM, 2006, pp. 262–271.

[36] T. Kim, J. Park, I. Kulida, and Y. Jang, "Concolic testing framework for industrial embedded software," in *2014 21st Asia-Pacific Software Engineering Conference*, vol. 2, Dec 2014, pp. 7–10.

[37] C. Zhang, Y. Yan, H. Zhou, Y. Yao, K. Wu, T. Su, W. Miao, and G. Pu, "Smartunit: Empirical evaluations for automated unit testing of embedded software in industry," in *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, 2018, pp. 296–305.

[38] P. Godefroid, A. Kiezun, and M. Y. Levin, "Grammar-based whitebox fuzzing," in *Programming Language Design and Implementation (PLDI)*, 2008.

[39] "Microsoft security risk detection," https://www.microsoft.com/en-us/security-risk-detection/.

[40] S. Bardin and P. Herrmann, "Structural testing of executables," in *International Conference on Software Testing, Verification and Validation (ICST)*, 2008.

[41] ——, "OSMOSE: automatic structural testing of executables," *Journal of Software Testing, Verification, and Reliability (STVR)*, vol. 21, no. 1, pp. 29–54, 2011.

[42] A. Franzén, A. Cimatti, A. Nadel, R. Sebastiani, and J. Shalev, "Applying smt in symbolic execution of microcode," in *Formal Methods in Computer Aided Design*, Oct 2010, pp. 121–128.

[43] G. Li, I. Ghosh, and S. P. Rajan, "Klover: A symbolic execution and automatic test generation tool for c++ programs," in *Computer Aided Verification*, G. Gopalakrishnan and S. Qadeer, Eds., 2011, pp. 609–615.

[44] M. Kim and Y. Kim, "Concolic testing of the multi-sector read operation for flash memory file system," in *Brazilian Symposium on Formal Methods*, 2009.

[45] Y. Park, S. Hong, M. Kim, D. Lee, and J. Cho, "Systematic testing of reactive software with non-deterministic events: A case study on lg electric oven," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 2, May 2015, pp. 29–38.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대 표 연 구 실 적 요 약 문 〉

대표연구실적 요약문			
연구업적 제목 (논문명/특허명)	Precise concolic unit testing of C programs using extended units and symbolic alarm filtering		
게재지	*학술지게재논문인 경우 저널명을 기술(Full Name)		
게재연도/등록(출원)연도	International Conference on Software Engineering (ICSE) 2018		
주관연구책임자 또는 참여연구원 성명	김문주	참여자수	3
SCI 등재 여부	해당없음	IF 및 피인용횟수 (SCI)	5 (Google scholar)
SCOPUS 등재 여부	해당없음	피인용횟수 (SCOPUS)	해당없음

요 약 문

Automated unit testing reduces manual effort to write unit test drivers/stubs and generate unit test inputs. However, automatically generated unit test drivers/stubs raise false alarms because they often over-approximate real contexts of a target function f and allow infeasible executions of f. To solve this problem, we have developed a concolic unit testing technique CONBRIO. To provide realistic context to f, it constructs an extended unit of f that consists of f and closely relevant functions to f. Also, CONBRIO filters out a false alarm by checking feasibility of a corresponding symbolic execution path with regard to f's symbolic calling contexts obtained by combining symbolic execution paths of f's closely related predecessor functions.

In the experiments on the crash bugs of 15 real-world C programs, CONBRIO shows both high bug detection ability (i.e. 91.0% of the target bugs detected) and high precision (i.e. a true to false alarm ratio is 1:4.5). Also, CONBRIO detects 14 new bugs in 9 target C programs studied in papers on crash bug detection techniques.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대표적 연구실적 사본 〉

2018 ACM/IEEE 40th International Conference on Software Engineering

Precise Concolic Unit Testing of C Programs using Extended Units and Symbolic Alarm Filtering

Yunho Kim
School of Computing, KAIST
Daejeon, South Korea
yunho.kim03@gmail.com

Yunja Choi
School of Computer Science and Engineering, Kyungpook Natl. Univ.
Daegu, South Korea
yuchoi76@knu.ac.kr

Moonzoo Kim
School of Computing, KAIST
Daejeon, South Korea
moonzoo@cs.kaist.ac.kr

ABSTRACT

Automated unit testing reduces manual effort to write unit test drivers/stubs and generate unit test inputs. However, automatically generated unit test drivers/stubs raise false alarms because they often over-approximate real contexts of a target function f and allow infeasible executions of f. To solve this problem, we have developed a concolic unit testing technique CONBRIO. To provide realistic context to f, it constructs an *extended unit* of f that consists of f and *closely relevant* functions to f. Also, CONBRIO filters out a false alarm by checking feasibility of a corresponding symbolic execution path with regard to f's *symbolic calling contexts* obtained by combining symbolic execution paths of f's closely related predecessor functions.

In the experiments on the crash bugs of 15 real-world C programs, CONBRIO shows both high bug detection ability (i.e. 91.0% of the target bugs detected) and high precision (i.e. a true to false alarm ratio is 1:4.5). Also, CONBRIO detects 14 new bugs in 9 target C programs studied in papers on crash bug detection techniques.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging;**

ACM Reference Format:
Yunho Kim, Yunja Choi, and Moonzoo Kim. 2018. Precise Concolic Unit Testing of C Programs using Extended Units and Symbolic Alarm Filtering. In *ICSE '18: ICSE '18: 40th International Conference on Software Engineering, May 27-June 3, 2018, Gothenburg, Sweden.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3180155.3180253

1 INTRODUCTION

Although unit testing is effective to detect SW bugs, field engineers have burden of manually generating test drivers/stubs and test inputs for each target unit. To reduce manual effort to generate test inputs, automated test generation has been applied (e.g., concolic testing have been applied to detect bugs in open source programs [2–4, 27, 31, 39] and industrial projects [6, 17, 23, 24, 33, 45] at system-level). Also, to reduce manual effort to generate unit test drivers/stubs, automated unit testing has been applied to open source programs [10, 14, 37] and large industrial SW [28].

A main drawback of the automated unit testing is a large number of *false alarms* raised by *infeasible unit executions* (i.e. unit executions that are infeasible at system-level). Infeasible unit executions occur generated due to inaccurate unit test drivers/stubs that over-approximate real contexts of a target unit (Sect. 2.4). This false alarm problem is a serious obstacle to apply automated unit testing in practice since field engineers would not like to spend time to manually filter out many false alarms.

To overcome this limitation, we have developed an automated unit testing framework CONBRIO (CONcolic unit testing with sym-Bolic alaRm fIltering using symbolic calling cOntexts) which operates in the following two stages:

1. To provide realistic context to a target function f, CONBRIO constructs an *extended unit* of f that consists of f and *closely relevant* functions to f which can filter out infeasible unit executions caused by symbolic stubs. The relevance of a function g to f is measured by the degree of dependency of f on g (Sect.3.2). Then, CONBRIO performs concolic execution of an extended unit of f.
2. To filter out false alarms by checking feasibility of a corresponding symbolic unit execution of f, CONBRIO generates *symbolic calling context* of f by combining symbolic paths of *closely relevant* predecessor functions of f in a static call graph.

As a result, CONBRIO detects bugs effectively and precisely because it enforces various and realistic executions of f through concolic execution of f with f's realistic contexts (i.e., with the functions closely relevant to f) and accurately filters out false alarms using f's symbolic calling contexts.

Note that it is important to construct an extended unit and symbolic calling context of f to contain *only* functions *closely relevant* to f since including more functions will enlarge symbolic search space and degrade unit testing effectiveness and efficiency. For example, at one extreme end, an extended unit may contain all successor functions of f and fail to detect bugs due to too large symbolic search space to explore. Also, symbolic calling context of f may contain symbolic execution paths of all predecessor functions of f up to `main` and fail to detect bugs. [1]

We have applied CONBRIO to 15 real-world C programs in SIR [11] and SPEC2006 [41] benchmarks and CONBRIO shows both

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSE '18, May 27-June 3, 2018, Gothenburg, Sweden
© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.
ACM ISBN 978-1-4503-5638-1/18/05…$15.00
https://doi.org/10.1145/3180155.3180253

[1] Generated symbolic calling context may *not* represent all feasible calling context of f due to the limitation of symbolic execution. Thus, a symbolic calling context becomes more difficult to satisfy by adding symbolic execution paths of more predecessor functions of f (i.e., via logical $\wedge$. See Sect. 3.5).

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

DART [16] generates symbolic unit test drivers (but not symbolic stubs) like SUT (Sect. 2.2) and test inputs for C programs. CONBOL [28] generates symbolic unit test drivers/stubs and test inputs targeting large-scale embedded C programs. DART and CONBOL generate symbolic unit test drivers without utilizing contexts of a target function f and may suffer many false alarms. [8]

Chakrabarti and Godefroid [9] developed a unit testing technique which statically partitions a static call graph using topological information and tests each partition as a unit through symbolic execution. This technique may suffer many false alarms because the obtained partitions may not represent groups of relevant functions due to insufficient information to generate partitions (i.e., using only topological information of a static call graph without semantic or dynamic information). Their tool is not publicly available and the paper does not report bug detection ability nor precision [9]. Tomb. et al [44] reported that interprocedural program analysis with deeper call depth bound raise fewer false alarms. However, they did not report how to set a proper call depth bound.

Recently, UC-KLEE [37] directly starts symbolic execution from a target function using lazy initialization [22]. Through the manual analysis of the thousands of alarms, the authors of UC-KLEE detected 67 new bugs in `BIND`, `openssl`, Linux kernel and its true to false alarm ratio is 1:5.7 on average. We could not directly compare CONBRIO with UC-KLEE because UC-KLEE is not publicly available and `BIND`, `openssl`, and Linux kernel (million lines of code) are too large to manually analyze alarms.

6.2 Random Method Sequences Generation Techniques for Object-Oriented Programs

Randoop [32] invokes a random sequence of public methods including constructors of a target method's class. MSeqGen [42] mines code bases to extract relevant method sequences of a target class under test and extends such method sequences with symbolic execution for high coverage. EvoSuite [13, 14] tests Java methods using search-based strategies with symbolic execution. TestFul [5] combines genetic algorithm and a local search to improve the speed of Java unit test generation. Garg et al. [15] improves Randoop by generating input test cases of the generated method sequence using concolic testing for C++ programs. These techniques may also suffer false alarms due to infeasible test inputs/method sequences generated. For example, Gross et al. [19] reported that Randoop raised 181 alarms without detecting any bug (i.e., all alarms were false ones) on five Java programs although the authors of Randoop reported that Randoop's true to false alarm ratio is 1:0.67 on 8 Java libraries and 6 .NET libraries on average [32]. Fraser et al. [14] reported that the statistically estimated true to false alarm ratios range from 1:0.6 to 1:4.2 in their experiments on randomly selected 100 projects hosted on sourceforge.net. Garg et al. [15] does not report detected bugs or false alarm ratios but branch coverage obtained using the proposed technique on eight programs (except `gnuchess` on which the authors reported nine new bugs and that a true to false alarm ratio was 1:1.0). In spite of the lack of explicit context information (e.g., class/object information) in C programs, CONBRIO detects bugs precisely in C programs (i.e., a true to false alarm ratio is 1:4.5 on average) while keeping high bug detection ability (i.e., 91.0% of the target bug detected on average).

The aforementioned papers report only bug detection precision (RQ2), not bug detection ability (RQ1), which makes fair comparison between these techniques and CONBRIO difficult. This is because these techniques may improve a true to false alarm ratio at the cost of missing bugs. Because of such trade-off between precision and recall of bug detection, we studied and reported both bug detection ability and precision.

6.3 Automated Unit Testing Techniques based on System Tests

Elbaum et al. [12] proposed a technique to generate unit tests from system tests; the technique captures program states before and after an invocation of a target function f to generate unit test inputs and oracles for f. OCAT [21] captures object instances during system executions and generates unit tests using Randoop with the captured object and the mutated object instances as seed objects. GenUTest [34] automatically generates unit tests and mock objects using captured method sequences during system testing. A limitation of these techniques is that the executions of the generated unit tests just replay the same behaviors [12, 34] (or similar behaviors [21]) of a target unit in already performed system testing (i.e., they are applicable to only regression testing of evolving software, not to a single version of software). Also, the aforementioned papers do not report bug detection ability nor precision.

7 CONCLUSION AND FUTURE WORK

We have presented an automated concolic unit testing technique CONBRIO which generates extended units to closely mimic the real contexts of a target function f and filters out false alarms using symbolic calling context formulas of f using relevant functions to f. Through the experiments, CONBRIO demonstrates both high bug detection ability (91.0% of all target bugs detected) and high bug detection precision (a true to false alarm ratio is 1:4.5). Furthermore, CONBRIO detects 14 new crash bugs in the latest versions of the nine target C programs studied in other papers on crash bug detection techniques.

As future work, to improve the precision of automated unit testing further, we plan to refine the function dependency metric by analyzing more semantic characteristic of target program executions. Also, we will improve bug detection ability further by applying an invasive software testing technique [26] which increases test coverage by utilizing diverse executions of mutated versions of a target program. In addition, we will utilize generated test cases to improve precision of mutation-based fault localization [20].

ACKNOWLEDGEMENT

This work is supported by Next-Generation Information Computing Development Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science and ICT (MSIT) (No. NRF-2017M3C4A7068175 and No. NRF-2017M3C4A7068177), and Basic Science Research Program through NRF funded by MSIT (NRF-2016R1A2B4008113), and Basic Science Research Program through NRF funded by the Ministry of Education (NRF-2017R1D1A1B03035851).

[8] DART [16] bypasses the false alarm issue by targeting public API functions of libraries which should work with all possible inputs.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대 표 연 구 실 적 요 약 문 〉

대표연구실적 요약문			
연구업적 제목 (논문명/특허명)	Invasive Software Testing: Mutating Target Programs to Diversify Test Exploration for High Test Coverage		
게재지	International Conference on Software Testing, Verification and Validation (ICST)		
게재연도/등록(출원)연도	2018		
주관연구책임자 또는 참여연구원 성명	김문주	참여자수	5
SCI 등재 여부	해당없음	IF 및 피인용횟수 (SCI)	4 (Google Scholar)
SCOPUS 등재 여부	해당없음	피인용횟수 (SCOPUS)	해당없음

요 약 문

Software testing techniques have advanced significantly over several decades; however, most of current techniques still test a target program as it is, and fail to utilize valuable information of diverse test executions on many variants of the original program in test generation. This paper proposes a new direction for software testing – Invasive Software Testing (IST). IST first generates a set of target program variants m_1, ⋯, m_n from an original target program p by applying mutation operations μ_1, ⋯, μ_n . Second, given a test suite T, IST executes m_1, ⋯, m_n with T and records the test runs which increase test coverage compared to p with T. Based on the recorded information, IST generates guideposts for automated test generation on p toward high test coverage. Finally, IST generates test inputs on p with the guideposts and achieves higher test coverage. We developed DEMINER which implements IST for C programs through software mutation and concolic testing. Further, we showed the effectiveness of DEMINER on three real-world target programs Busybox-ls, Busybox-printf, and GNU-find. The experiment results show that the amount of the improved branch coverage by DEMINER is 24.7% relatively larger than those of the conventional concolic testing techniques on average.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대표적 연구실적 사본 〉

2018 IEEE 11th International Conference on Software Testing, Verification and Validation

Invasive Software Testing: Mutating Target Programs to Diversify Test Exploration for High Test Coverage

Yunho Kim*, Shin Hong†, Bongseok Ko*, Duy Loc Phan*, Moonzoo Kim*
*School of Computing, KAIST, South Korea
yunho.kim03@gmail.com, bsko5006@gmail.com, duyloc_1503@kaist.ac.kr, moonzoo@cs.kaist.ac.kr
†School of CSEE, Handong Global University, South Korea
hongshin@handong.edu

***Abstract*—Software testing techniques have advanced significantly over several decades; however, most of current techniques still test a target program *as it is*, and fail to utilize valuable information of *diverse test executions on many variants* of the original program in test generation.**

This paper proposes a new direction for software testing – *Invasive Software Testing (IST)*. IST first generates a set of target program variants $m_1, ..., m_n$ from an original target program p by applying mutation operations $\mu_1, ..., \mu_n$. Second, given a test suite T, IST executes $m_1, ..., m_n$ with T and records the test runs which increase test coverage compared to p with T. Based on the recorded information, IST generates *guideposts* for automated test generation on p toward high test coverage. Finally, IST generates test inputs on p with the guideposts and achieves higher test coverage. We developed DEMINER which implements IST for C programs through software mutation and concolic testing. Further, we showed the effectiveness of DEMINER on three real-world target programs `Busybox-ls`, `Busybox-printf`, and `GNU-find`. The experiment results show that the amount of the improved branch coverage by DEMINER is 24.7% relatively larger than those of the conventional concolic testing techniques on average.

***Index Terms*—automated test generation, concolic testing, mutation analysis, test coverage**

I. INTRODUCTION

Software testing has been a de-facto standard method to assess and improve software quality. Although software testing techniques have advanced significantly over several decades, most of them (either manual or automated ones) still test a target program *as it is* without modifying the target program. In contrast, other engineering disciplines such as mechanical engineering have used *invasive testing* (or destructive testing) as a standard technique to obtain information on a target object (i.e., analyzing a target object by inducing physical forces on it) which cannot be provided by a non-invasive testing.

Note that a software program has the following ideal characteristics for invasive testing:

- it is almost free to make copies of a software program.
- it is easy to modify/destroy a software program (in a source code or binary level).

However, owing to high complexity of software, it is non-trivial to re-interpret/re-analyze various dynamic information obtained from many modified variants of the original target program (e.g., what conclusion can we make on an original program p by observing crashes on a variant m_1 of p with a test case t_1? For example, crash of m_1 with t_1 does not necessarily mean that p will crash with t_1). Consequently, software testing so far has failed to improve testing effectiveness further by utilizing useful information from diverse test executions on many variants of a target program.

To resolve the aforementioned problem, this paper proposes a new direction for software testing: *Invasive Software Testing (IST)*. This technique consists of the following three stages:

1) extract/learn useful information of an original target program p for high test coverage through the diverse test exploration of various variants of p
2) create *guideposts* by using the extracted information, which can guide test generation for high test coverage
3) generate test executions on p following/satisfying the guideposts inserted in p

A core idea of IST is to create *guideposts* by learning knowledge on an original target program p from diverse test executions of many mutated versions $m_1, \ldots, m_n$ of p, which can effectively lead test generation to cover various executions of p that achieve high test coverage.

We developed DEMINER (gui**DE**ed test generation using **M**utat**I**o**N** **E**xplo**R**ation) which realizes IST to improve test coverage. DEMINER operates as follows:

1) DEMINER generates target program variants $m_1, \ldots, m_n$ from an original target program p by applying mutation $\mu_1, \ldots, \mu_n$, respectively.
2) given a test suite T, DEMINER executes $m_1, \ldots, m_n$ with T and records test runs covering new lines not covered on p with T and corresponding mutation operator instances.
3) Based on the recorded information, DEMINER constructs *guideposts* to guide test generation on p to improve test coverage.
4) DEMINER applies concolic testing on p with an inserted guidepost to generate new test executions that follow the guideposts and achieve high test coverage.

978-1-5386-5012-7/18/$31.00 ©2018 IEEE
DOI 10.1109/ICST.2018.00032

IEEE computer society

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

because they iterate over a symbolic input string `format` until they reach any specified character (i.e., '.', '*', or '\0'). Therefore, conventional concolic testing keeps increasing the loop bound and fails to cover Lines 202–203 within a given time bound.

Note that DEMINER covers these hard-to-cover lines as follows. DEMINER generates a mutant m which mutates a variable `fmt_length` to another variable `precision`. One of the mutant executions of m covers Lines 202–203 and the monitoring probe for m (i.e., `probe(format[precision])` at right before Line 169) reports 'X' as a value of the mutated target expression. Using the reported value 'X', DEMINER inserts a guidepost at Line 168 and guided concolic testing of DEMINER tries to satisfy the guidepost condition with high priority instead of negating branches of the loops inside `strstr` and `strchr` at Lines 172–173 (see Section III-D). Therefore, DEMINER effectively covers Lines 202–203.

Furthermore, for `GNU-find`, DEMINER covers 16 lines that seems not reachable by the conventional concolic testing by any means. We ran each of the conventional concolic testing techniques using DFS, RND and CFG for one week (i.e., 168 hours) per each initial test case (i.e., execution time is 120 weeks in total). Then, we found that there are 16 lines which are covered by DEMINER, but they were never covered by any test executions by conventional concolic techniques with 120 weeks of the testing time. DEMINER can cover these 16 lines successfully because the guideposts prune the search space that is not relevant to cover new lines, and effectively guide concolic testing toward execution paths that reach unreached lines observed from diverse coverage-increasing mutant executions.

B. Application of Mutation Analysis

Program mutation has been a popular method for evaluating how given test cases detect subtle program changes. Traditional research on software mutation [22] mainly focuses on evaluating bug finding effectiveness of a test suite by measuring how many mutants the test suite can kill.

Fraser and Zeller [23] presents a search-based unit test generation technique that targets mutants as a way to generate diverse unit tests. The technique directs test case generation toward finding output and coverage differences between a target program and its mutants. Both our approach and that by Fraser and Zeller [23] are common in that both analyze mutant executions. The difference is that in our approach, the dynamic information on mutants is used for inferring internal conditions of a target program to increase test coverage, rather than measuring the difference between the target program and a mutant.

Program mutation has also been used to generate and evaluate test oracles. Mutation analysis is used to examine which properties are invariants of the correct program. Fraser and Zeller [23] utilizes mutation analysis to infer test oracles from the mutant execution information. Jahangirova et al. [24] presents a method to assess and improve the quality of a given test oracle by utilizing program mutation. Recently, mutation analysis has been used to precisely localize a fault in a program. Mutation-based fault localization [25]–[28] locates a fault in the target program code by observing how the behaviors of the faulty program change according to the program code chances.

A few research work have focused on mutating program code or runtime states to derive diverse executions for better dynamic analyses [29], [30]. These techniques utilize values and execution paths observed from mutated program executions directly during the analyses of the original program without validating whether these observations also hold for the original program. Thus, these techniques may produce unsound analysis results because mutated program executions may be infeasible on the original program.

Our approach alleviates this false positive problem by generating only feasible test inputs by leveraging concolic testing techniques with the discovered knowledge from the mutation analyses.

VII. CONCLUSION

This paper presents DEMINER which is an automated test generation technique that realizes the invasive software testing paradigm by utilizing information from diverse mutant executions. We demonstrated that DEMINER can effectively increase test coverage by applying DEMINER to three real-world C programs.

We plan to extend DEMINER in the following directions. First, to enhance the knowledge discovery, we will improve DEMINER to employ more mutation operators, including statement-level and higher-order mutation operators. Second, to learn the converage-increasing conditions more efficiently and effectively, we will utilize the automated unit test generation technique using concolic testing [31] which can generate the coverage-increasing conditions at an entry of a function. Third, we will leverage program invariant inference techniques, such as Daikon [32], to generate various kinds of guideposts from mutant execution information. In addition, we will explore what kinds of guidepost condition structures are effective for capturing useful knowledge from mutant executions. Finally, we will apply DEMINER to more real-world programs to show that DEMINER is generally effective in finding unknown bugs in real-world programs.

ACKNOWLEDGMENTS

This research was supported by Next-Generation Information Computing Development Program (No. 2017M3C4A7068177, No. 2017M3C4A7068179), Basic Science Research Program (No. 2016R1A2B4008113, No. 2017R1C1B1008159) through the National Research Foundation (NRF) funded by the Ministry of Science and ICT of Korea, Basic Science Research Program (No. 2017R1D1A1B03035851) through NRF funded by the Ministry of Education of Korea, and KAIST High Risk High Return Project (HRHRP).

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대 표 연 구 실 적 요 약 문 〉

대표연구실적 요약문			
연구업적 제목 (논문명/특허명)	MUSEUM:Debugging real-world multilingual programs using mutation analysis		
게재지	Information and Software Technology (IST)		
게재연도/등록(출원)연도	2017		
주관연구책임자 또는 참여연구원 성명	김문주	참여자수	7
SCI 등재 여부	Y	IF 및 피인용횟수 (SCI)	IF: 2.627, 피인용: 13 (JCR2017, Google scholar)
SCOPUS 등재 여부	Y	피인용횟수 (SCOPUS)	9

요 약 문

Context: The programming language ecosystem has diversified over the last few decades. Non-trivial programs are likely to be written in more than a single language to take advantage of various control/data abstractions and legacy libraries.

Objective: Debugging multilingual bugs is challenging because language interfaces are difficult to use correctly and the scope of fault localization goes beyond language boundaries. To locate the causes of real-world multilingual bugs, this article proposes a mutation-based fault localization technique (MUSEUM).

Method: MUSEUM modifies a buggy program systematically with our new mutation operators as well as conventional mutation operators, observes the dynamic behavioral changes in a test suite, and reports suspicious statements. To reduce the analysis cost, MUSEUM selects a subset of mutated programs and test cases.

Results: Our empirical evaluation shows that MUSEUM is (i) effective: it identifies the buggy statements as the most suspicious statements for both resolved and unresolved non-trivial bugs in real-world multilingual programming projects; and (ii) efficient: it locates the buggy statements in modest amount of time using multiple machines in parallel. Also, by applying selective mutation analysis (i.e., selecting subsets of mutants and test cases to use), MUSEUM achieves significant speedup with marginal accuracy loss compared to the full mutation analysis.

Conclusion: It is concluded that MUSEUM locates real-world multilingual bugs accurately. This result shows that mutation analysis can provide an effective, efficient, and language semantics agnostic analysis on multilingual code. Our light-weight analysis approach would play important roles as programmers write and debug large and complex programs in diverse programming languages.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대표적 연구실적 사본 〉

Information and Software Technology 82 (2017) 80–95

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof

MUSEUM: Debugging real-world multilingual programs using mutation analysis

Shin Hong[a], Taehoon Kwak[b], Byeongcheol Lee[c,*], Yiru Jeon[b], Bongseok Ko[c], Yunho Kim[b], Moonzoo Kim[b]

[a] Handong Global University, 558 Handong-ro, Buk-gu, Pohang, Gyeongbuk, 37554, South Korea
[b] Korea Institute of Science and Technology, 291 Daehak-ro, Yuseong-gu, Daejeon, 34141, South Korea
[c] Gwangju Institute of Science and Technology, 123 Cheomdangwagi-ro, Buk-gu, Gwangju, 61005, South Korea

ARTICLE INFO

Article history:
Received 29 July 2016
Revised 12 October 2016
Accepted 13 October 2016
Available online 14 October 2016

Keywords:
Debugging
mutation analysis
Language interoperability
Foreign function interface

ABSTRACT

Context: The programming language ecosystem has diversified over the last few decades. Non-trivial programs are likely to be written in more than a single language to take advantage of various control/data abstractions and legacy libraries.

Objective: Debugging multilingual bugs is challenging because language interfaces are difficult to use correctly and the scope of fault localization goes beyond language boundaries. To locate the causes of real-world multilingual bugs, this article proposes a mutation-based fault localization technique (MUSEUM).

Method: MUSEUM modifies a buggy program systematically with our new mutation operators as well as conventional mutation operators, observes the dynamic behavioral changes in a test suite, and reports suspicious statements. To reduce the analysis cost, MUSEUM selects a subset of mutated programs and test cases.

Results: Our empirical evaluation shows that MUSEUM is (i) effective: it identifies the buggy statements as the most suspicious statements for both resolved and unresolved non-trivial bugs in real-world multilingual programming projects; and (ii) efficient: it locates the buggy statements in modest amount of time using multiple machines in parallel. Also, by applying selective mutation analysis (i.e., selecting subsets of mutants and test cases to use), MUSEUM achieves significant speedup with marginal accuracy loss compared to the full mutation analysis.

Conclusion: It is concluded that MUSEUM locates real-world multilingual bugs accurately. This result shows that mutation analysis can provide an effective, efficient, and language semantics agnostic analysis on multilingual code. Our light-weight analysis approach would play important roles as programmers write and debug large and complex programs in diverse programming languages.

© 2016 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Modern software systems are written in multiple programming languages to reuse legacy code and leverage the languages best suited to the developers' needs such as performance and productivity. A few languages cover the most use in part due to open source libraries and legacy code while many languages exist for niche uses [30]. This ecosystem encourages developers to write a *multilingual program* which is a non-trivial program written in multiple languages. Correct multilingual programs are difficult to write due to the complex language interfaces such as Java Native Interface (JNI) and Python/C that require the programs to respect a set of thousands of interface safety rules over hundreds of application interface functions [22,26]. Moreover, if a bug exists at interactions of code written in different languages, programmers have to understand the cause-effect chains across language boundaries [21].

Despite the advance of automated testing techniques for complex real-world programs, debugging multilingual bugs (e.g., a bug

* Corresponding author.
E-mail addresses: hongshin@handong.edu (S. Hong), thkwak@kaist.ac.kr (T. Kwak), byeong@gist.ac.kr (B. Lee), podray@kaist.ac.kr (Y. Jeon), bsk@gist.ac.kr (B. Ko), kimyunho@kaist.ac.kr (Y. Kim), moonzoo@cs.kaist.ac.kr (M. Kim).

http://dx.doi.org/10.1016/j.infsof.2016.10.002
0950-5849/© 2016 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

that of the fault localization with MPS(SD,1) and TM (the third row) on one machine. The numbers in the second row are calculated by multiplying 30 to the time cost in Table 2. MUSEUM with the MPS(SD,1) and TM selection strategies consumes only 3.8% of the time cost with all mutants and all test cases for the eight target bugs on average (see the last column of the last row). Thus, this result confirms that the selection strategy can effectively reduce the time cost of MUSEUM as the number of the mutant executions is reduced.[5]

9. Discussions

9.1. Advantages of the mutation-based fault localization for real-world multilingual programs

For large real-world programs, it is challenging to build test cases that exercise diverse execution paths because it is non-trivial to understand and control a target program. Also, generating diverse test cases for multilingual programs has additional burden to learn and satisfy safety rules on language interfaces. Thus, multilingual programs are often developed with only simple test cases, which makes the SBFL techniques fail to accurately localize the eight real-world multilingual bugs (Table 3).

For example, the statement coverages of the test suites used for Bug2 and Bug3 are around 85% and 86% and the SBFL techniques localize these bugs somehow precisely (i.e., the suspiciousness rank of Bug2 and Bug3 are 4 and 5, respectively). However, the statement coverages of the test suites used for Bug1, Bug4, Bug5, Bug6, and Bug8 are around 1%, 22%, 24%, 19%, and 11% and the accuracy of the SBFL techniques for these bugs are very low (Table 3). In contrast, MUSEUM can alleviate this limitation by achieving the effect of diverse test cases through the diverse mutants with limited test cases. Thus, MUSEUM can be a promising technique for debugging complex real-world multilingual programs.

9.2. Effectiveness of the new mutation operators for localizing multilingual bugs

The experiment results show that the new mutation operators are effective to generate informative mutants (i.e., partial fix mutants) to localize multilingual bugs. For Bug1, Bug4, and Bug8, only the new mutation operators generate partial fix mutants. For Bug5 and Bug7, the new mutation operators and the existing ones generate partial fix mutants. For Bug2, Bug3 and Bug6, only existing mutation operators generate partial fix mutants.

To assess the impact of the new mutation operators on fault localization, we ran MUSEUM for Bug1, Bug4, Bug8, Bug5 and Bug7 without the new mutation operators. For Bug1, Bug4 and Bug8, the suspiciousness ranking of the faulty line becomes significantly low (1,737 (89.6%) for Bug1, 117 (62.9%) for Bug4, and 3,061 (61.2%) for Bug8). For Bug5, the ranking of the faulty line changes from the eighth to the ninth and the faulty line is not anymore the most suspicious statement. For Bug7, the ranking of the faulty line remains unchanged.

9.3. High accuracy with low runtime cost through selective mutation analysis

The selective mutation analysis for MUSEUM can achieve high fault localization accuracy with significantly reduced runtime cost (e.g., MPS(SD,1) and TM can reduce the runtime cost up to 96% and identifies the faulty statements as the most suspicious ones) (Section 8.3.1). Also, more mutants and test cases can increase the fault localization accuracy with the selective mutation analysis (Fig. 2).

Thus, MUSEUM should start with the mutants and test cases selected by a selection strategy (e.g., MPS(SD,1) and TM). Then, MUSEUM can add more mutants and test cases by relaxing the parameter of the selection strategy or changing the selection strategy. In this way, MUSEUM can achieve high fault localization accuracy with low runtime cost first and then increase the fault localization accuracy gradually within the given time budget.

10. Conclusion and future work

We have presented MUSEUM which localizes bugs in complex real-world multilingual programs in a language semantics agnostic manner through mutation analyses. The experiments and the case studies show that MUSEUM accurately locates the faulty statements for all non-trivial Java/C bugs. Also, we show that the accuracy of fault localization for multilingual programs can be increased by adding new mutation operators relevant with language interface constraints. Finally, our selection strategies over mutants and test cases significantly reduce the analysis time with marginal accuracy loss.

As future work, we will add more mutation operators targeting multilingual features and higher-order mutation operators to reduce equivalent mutants and generate useful mutants. Also, we will apply MUSEUM to an interactive debugger such as Blink [21] to maximize the debugging effectiveness. Finally, we will investigate how to utilize MUSEUM to improve program repair and search-based program analysis for multilingual programs.

Acknowledgements

This work is supported by the MSIP, Korea, under the ITRC support program (IITP-2016-H8501161012000l002) supervised by the IITP, the NRF grants (NRF-2014K1A3A1A09063167, NRF-2015R1C1A1A01055259, NRF-2015R1C1A1A01052876, and NRF-2016R1A2B4008113) funded by the Korea government (MSIP), the IITP grant programs funded by the MSIP (Research and Development of Dual Operating System Architecture with High-Reliable RTOS and High-Performance OS [No. R0101-15-0081] and High Performance Big Data Analytics Platform Performance Acceleration Technologies Development [No. R0190-16-2012]), and the GIST Research Institute (GRI) grant in 2016.

References

[1] PIT v0.33 - mutation testing tool for Java, (http://pitest.org).
[2] JDK-4804447: JNI get(type)arrayelements fail with zero length arrays, 2003, (https://bugs.openjdk.java.net/browse/JDK-4804447).
[3] Azureus-commitlog: ListView.java, 2008, (http://sourceforge.net/p/azureus/mailman/message/18318135/).
[4] Java-GNOME Avoid segfault lurking in GtkSpell library, 2009, (https://openhub.net/p/java-gnome-gstreamer/commits/167384488).
[5] JNI Local Reference Changes in ICS, 2011, (Android Developers Blog, http://android-developers.blogspot.com/2011/11/jni-local-reference-changes-in-ics.html).
[6] Xerial SQLite-JDBC, Issue 16: DDL statements return result other than 0, 2012, (https://bitbucket.org/xerial/sqlite-jdbc/issue/16).
[7] Xerial SQLite-JDBC, Issue 36: Calling PreparedStatement.clearParameters() after a ResultSet is opened, causes subsequent calls to the ResultSet to return null, 2013, (https://bitbucket.org/xerial/sqlite-jdbc/issue/36).
[8] Firefox Bug 958706 - Don't hide JNI exceptions, 2014, (https://bugzilla.mozilla.org/show_bug.cgi?id=958706).
[9] Eclipse SWT bug419729: Native crash in org.eclipse.swt.internal.gtk.OS._gtk_widget_show, 2015, (https://bugs.eclipse.org/bugs/show_bug.cgi?id=419729).
[10] JNI APIs and developer guides, 2015, (http://docs.oracle.com/javase/8/docs/technotes/guides/jni).

[5] The ratio in Table 10 can be different from the ratio in Table 6 because the time cost of MUSEUM involves mutant generations, data processing and other operational steps in addition to mutant executions (also execution time of a mutant can be different depending on the mutant and the test case used).

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대 표 연 구 실 적 요 약 문 〉

대표연구실적 요약문			
연구업적 제목 (논문명/특허명)	MUSIC: Mutation Analysis Tool with High Configurability and Extensibility		
게재지	Mutation Workshop		
게재연도/등록(출원)연도	2018		
주관연구책임자 또는 참여연구원 성명	김문주	참여자수	3
SCI 등재 여부	해당없음	IF 및 피인용횟수 (SCI)	2 (Google scholar)
SCOPUS 등재 여부	해당없음	피인용횟수 (SCOPUS)	해당없음

요 약 문

Although mutation analysis is important for various software analysis tasks, there exist few practical mutation tools for C programs. We have developed MUSIC (MUtation analySIs tool with high Configurability and extensibility) which generates mutants for modern complex real-world C programs. MUSIC provides various mutation operators including 10 new mutation operators such as string and function call mutation operators as well as 63 conventional mutation operators. Also, MUSIC supports a user to create a new mutation operator easily. Furthermore, MUSIC can select a domain and a range of a mutation operator for various purposes.

We have applied Milu, Proteum, and MUSIC to Siemens benchmark programs and a modern real-world C program cURL, and compared them in terms of applicability and a number of stillborn (i.e., syntactically illegal) mutants generated. In the experiment, MUSIC successfully generates mutants without any stillborn mutants.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 대표적 연구실적 사본 〉

2018 IEEE International Conference on Software Testing, Verification and Validation Workshops

MUSIC: Mutation Analysis Tool with High Configurability and Extensibility

Duy Loc Phan, Yunho Kim and Moonzoo Kim
School of Computing, KAIST
South Korea
Email: duyloc_1503@kaist.ac.kr, yunho.kim03@gmail.com, moonzoo@cs.kaist.ac.kr

***Abstract*—Although mutation analysis is important for various software analysis tasks, there exist few practical mutation tools for C programs. We have developed MUSIC (MUtation analySIs tool with high Configurability and extensibility) which generates mutants for modern complex real-world C programs. MUSIC provides various mutation operators including 10 new mutation operators such as string and function call mutation operators as well as 63 conventional mutation operators. Also, MUSIC supports a user to create a new mutation operator easily. Furthermore, MUSIC can select a domain and a range of a mutation operator for various purposes.**

We have applied Milu, Proteum, and MUSIC to Siemens benchmark programs and a modern real-world C program cURL, and compared them in terms of applicability and a number of stillborn (i.e., syntactically illegal) mutants generated. In the experiment, MUSIC successfully generates mutants without any stillborn mutants.

***Index Terms*—Mutation analysis, Practical mutation tool, C programs**

1. Introduction

Mutation analysis is important for various software analysis tasks such as evaluating quality of a test suite, fault localization [1]–[4], and test generation [5], [6]. However, there exist few practical mutation tools for C programs. Existing mutation tools for C programs often fail to generate useful mutants of a modern real-world C program. For example, Proteum [7] often fails to generate mutants for a modern C program because it does not support a recent C standard later than C89. For another example, Milu [8] generates many stillborn (i.e., syntactically illegal) mutants due to incorrect handling of types including `typedef`, `enum`, `const` type qualifier and an array type.

We have developed MUSIC (MUtation analySIs tool with high Configurability and extensibility) to generate mutants for modern complex real-world C programs that consist of multiple source files with complex compilation commands. MUSIC implements 63 mutation operators defined by Agrawal et al. [9] and 10 new mutation operators for strings and function calls to generate diverse mutants. MUSIC is designed to be extensible for a user to easily make a new mutation operator.

One salient feature of MUSIC is its fine-grained configuration of mutant generation, which can satisfy various purposes of mutation analysis. In other words, MUSIC allows a user to specify a target domain and a range of a mutation operator and a target scope of mutation. For example, MUSIC can apply arithmetic mutation operator (OAAN) to only one of {+,-} and mutate it to only * between Line 100 and Line 200 of `target.c`.

We have applied Milu, Proteum, and MUSIC to Siemens benchmark programs [10] and a modern large program cURL [11]. We evaluate these tools in terms of applicability and a number of stillborn mutants generated. For both Siemens benchmarks and cURL, MUSIC successfully generates mutants without any manual modification of the target programs and it generates no stillborn mutant. In contrast, Proteum requires manual source code modification to generate mutants for Siemens benchmarks and fails to generate mutants for cURL. Milu generates many stillborn mutants (34.18% and 75.31% of the mutants for Siemens benchmarks and cURL were syntactically illegal, respectively).

Our contributions are as follows:

1) We have developed MUSIC which is highly configurable and easy to extend for various mutation analysis purposes targeting modern complex real-world C programs.
2) MUSIC provides an extensive set of 73 mutation operators including 63 existing operators defined in Agrawal et al. [9] and 10 new operators which do not generate stillborn mutants.
3) We have performed an experiment to evaluate Milu, Proteum and MUSIC on Siemens benchmarks and a modern real-world C program cURL and demonstrated that MUSIC has high applicability and generates no stillborn mutant.

Sect. 2 explains MUSIC. Sect. 3 reports the experiment result on Siemens benchmarks and cURL. Sect. 4 concludes this paper with future work.

2. MUSIC Technique

We have implemented MUSIC based on the modern compiler framework Clang/LLVM 4.0 [12]. MUSIC is written in 9,000 lines of C++ code, which consists of 178 header

0-7695-6432-1/18/$31.00 ©2018 IEEE
DOI 10.1109/ICSTW.2018.00026

IEEE computer society

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

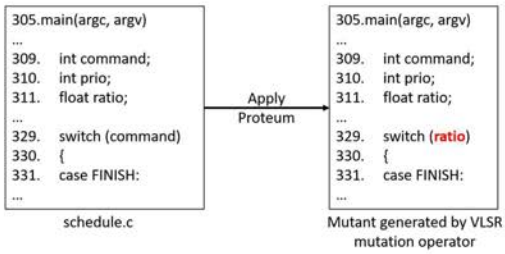


Figure 8. Example of stillborn mutants generated by Proteum

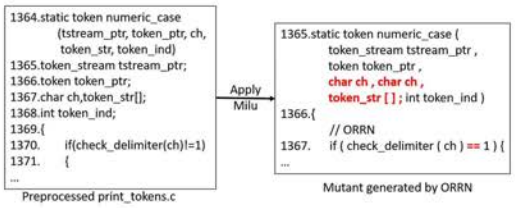


Figure 9. Example of stillborn mutants generated by Milu

Milu, Proteum and MUSIC. Milu supports only 28 mutation operators and usually generates much less mutants than Proteum (78 mutation operators) and MUSIC (73 mutation operators). As a result, on average, Milu, Proteum, and MUSIC generate 2273.1, 4896.9 and 6469.9 mutants on Siemens benchmark programs, respectively.

For Siemens benchmark programs, MUSIC generates 1573.0 more mutants than Proteum, on average. The difference is mainly caused by the way CCCR (Constant for Constant Replacement) and CCSR (Constant for Scalar Replacement) are implemented in each tool. MUSIC's CCCR and CCSR mutate a constant and a variable to a constant in a target program including a global constant as defined in Agrawal [9], respectively. However, Proteum's CCCR and CCSR do *not* mutate a constant or a variable to a global constant.

For cURL, Milu and MUSIC generates 41,470 and 1,035,088 mutants respectively.

3.2.2. Number of Stillborn Mutants. The fifth column of Table 1 shows that MUSIC generates no stillborn mutants for Siemens benchmark programs. In contrast, 33.18% and 3.68% of all mutants generated by Milu and Proteum are syntactically illegal and uncompilable, respectively (the sixth column of Table 1).

An example of a stillborn mutant generated by Proteum is shown in Fig. 8. Proteum's VLSR (Local Scalar Variable Replacement) mutates a condition variable `command` on Line 329 to a floating variable `ratio`. Since switch statement cannot take a floating variable, the generated mutant is syntactically illegal.

Fig. 9 shows an example of a stillborn mutant generated by Milu. When applying Milu to mutate function `numeric_case` in `printtokens`, all 170 generated mutants were syntactically illegal due to the syntax errors occurred in function definition: redefinition of parameter `ch` and inclusion of semicolon in the list of function parameters.

For cURL, MUSIC generates no stillborn mutant while Milu generates 31,232 ones (i.e., 75.31% of the generated mutants are syntactically illegal). We found that this large number of stillborn mutants is caused by incorrect handling of types including `typedef`, `enum`, `const` type qualifier and an array type.

4. Conclusion and Future Work

We have presented MUSIC which can generate diverse mutants for various mutation analysis purposes on modern real-world C programs. Through the experiments on Siemens benchmark programs and cURL, we have demonstrated that MUSIC has high applicability and generates no stillborn mutant.

As future work, we will apply MUSIC to more large C projects to evaluate its applicability further. Also, since mutation analysis is actively used for various software analysis tasks, we plan to provide more diverse mutation operators in MUSIC. Furthermore, we apply MUSIC to automated unit-test generation for C programs [13], [14] to enhance bug detection ability and test coverage by diversifying unit test executions through mutation analysis.

MUSIC is publicly available at https://github.com/swtv-kaist/MUSIC

Acknowledgments

This research was supported by Next-Generation Information Computing Development Program (No. 2017M3C4A7068177), Basic Science Research Program (No. 2016R1A2B4008113) through the National Research Foundation (NRF) funded by the Ministry of Science and ICT of Korea, and Basic Science Research Program (No. 2017R1D1A1B03035851) through NRF funded by the Ministry of Education of Korea.

References

[1] M. Papadakis and Y. L. Traon, "Metallaxis-FL: mutation-based fault localization," *Softw. Test., Verif. Reliab.*, vol. 25, no. 5-7, pp. 605–628, 2015.

[2] S. Moon, Y. Kim, M. Kim, and S. Yoo, "Ask the mutants: Mutating faulty programs for fault localization," in *2014 IEEE Seventh International Conference on Software Testing, Verification and Validation*, March 2014, pp. 153–162.

[3] S. Hong, B. Lee, T. Kwak, Y. Jeon, B. Ko, Y. Kim, and M. Kim, "Mutation-based fault localization for real-world multilingual programs (t)," in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, Nov 2015, pp. 464–475.

[4] S. Hong, T. Kwak, B. Lee, Y. Jeon, B. Ko, Y. Kim, and M. Kim, "MUSEUM: Debugging real-world multilingual programs using mutation analysis," *Information and Software Technology*, vol. 82, pp. 80 – 95, 2017.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

기타 실적 요약문	
연구실적 유형	언론보도 발표
연구업적 제목	현대모비스 차량 소프트웨어 검증 시스템 MAIST 개발
비고	**현대모비스와의 산학연구를 통해 개발한 소프트웨어 검증 시스템 MAIST를 적용하여 현대모비스 차량 SW 검증 업무량을 절반 이상 단축하고 신문 기사화함**

인공지능 소프트웨어 검증시스템 '마이스트(MAIST)' … 업무효율 2배 향상

현대모비스와 공동으로 개발한 마이스트는 개발자 대신 체계적이고 자동화된 방법으로 소프트웨어 검증작업을 수행하는 시스템이다. 마이스트는 연구원들이 설계한 알고리즘을 바탕으로 소프트웨어의 모든 가능한 동작을 체계적으로 검증한다. 기존 수작업으로 이뤄지던 소프트웨어 검증 업무를 자동화한 것이다.

현대모비스는 마이스트를 통합형 차체제어시스템(IBU)과 써라운드뷰모니터링 시스템(SVM) 검증에 시범 적용한 결과, IBU와 SVM의 소프트웨어 검증 업무의 각 53%, 70%를 마이스트가 처리했다고 밝혔다.

현대모비스 플랫폼 소프트웨어 개발실장 이승용 이사는 "마이스트가 검증 업무의 50~70%를 대신하면서 연구원들은 더 창의적인 환경에서 알고리즘 설계 역량을 강화하고 급속도로 증가하는 차량용 소프트웨어 개발에 신속 정확하게 대응할 수 있게 됐다"고 말했다.

현대모비스는 2018년 하반기부터 마이스트를 소프트웨어가 탑재되는 전장 전 부품으로 확대 적용한다. 제동과 조향 등의 핵심부품은 물론 자율주행 · 첨단 운전자 지원시스템(ADAS) · 친환경 등 미래차 연구 전 부문으로 확대한다. 글로벌 소프트웨어 연구기지인 인도연구소에도 적용할 방침이다.

현대모비스는 마이스트가 업무 효율을 2배 이상 높여 소프트웨어 개발 기간을 단축하고 검증의 정확도를 한층 더 높여 자율주행과 커넥티비티로 대표되는 전장부품의 설계 역량 강화와 품질 향상에 크게 기여할 것으로 기대하고 있다.

국가연구개발 보고서원문 성과물 전담기관인 한국과학기술정보연구원에서 가공·서비스 하는 연구보고서는 동의 없이 상업적 및 기타 영리목적으로 사용할 수 없습니다

〈 기타 실적 증빙자료 〉

연합뉴스 기사: https://www.yna.co.kr/view/AKR20180720158800003

